Degree project

UNIVERSITY OF SKÖVDE

Encrypted Client Hello, Balancing Privacy Enhancements with Security Implications

Bachelor Degree Project in Informatics

Final Submission 30 credits

Spring term 2024

Students: Paulius Kartavcevas & Luka Maksimovic

Supervisor: Jonas Ingemarsson

Examiner: Dr. Oskar MacGregor

LIST OF ABBREVIATIONS

ABBREVIATION	FULL TEXT
ACK	Acknowledge, part of the TCP handshake
ACME	Automated Certificate Management Environment
ALPN	Application Layer Protocol Negotiation
BIND	Berkeley Internet Name Domain
CDN	Content Delivery Network
DDoS	Distributed Denial-of-Service
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
DNSSEC	Domain Name System Security Extension
DoH	DNS over HTTPS
DPI	Deep Packet Inspection
ECH	Encrypted Client Hello
ESNI	Encrypted Server Name Indication
GfW	Great Firewall
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IETF	Internet Engineering Task Force
IP	Internet Protocol
ISP	Internet Service Provider
IT	Information Technology
MitM	Man-in-the-Middle
nVME	non-Volatile Memory Express
OS	Operating System
QoS	Quality of Service
RFC	Request For Comment
RPZ	Response Policy Zone
SCTP	Stream Control Transmission Protocol

SDGs	Sustainable Development Goals
SMTP	Simple Mail Transfer Protocol
SNI	Server Name Indication
SSL	Secure Sockets Layer
SYN	Synchronise, part of the TCP handshake
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTL	Time To Live
UDP	User Datagram Protocol
UFW	Uncomplicated Firewall
URL	Uniform Resource Locator
VoIP	Voice over Internet Protocol
VPN	Virtual Private Network
VPS	Virtual Private Server

PREFACE

This section is dedicated towards the contributors for this research project. We want to thank our supervisor Jonas Ingemarsson who provided practical guidance for conducting this research as well as our examiner Dr. Oskar MacGregor who aided us in the evaluation process and provided formal guidelines. We also want to thank all professors who aided in the forming process of the research mainly, Dr. Martin Lundgren and Mohammad Abbas Khan Abasi. We also appreciate our peers for aiding in the peer reviewing process. Lastly but not least, we also want to extend our gratitude to our related family for their support.

ABSTRACT

With the addition of *Transport Layer Security* (TLS) 1.3 extension *Encrypted Client Hello* (ECH), new challenges are present for network- and system administrators in relation to web content moderation. Notable concerns have been issued for ECH circumventing content filtering efforts as ECH introduces new functionality to cover up previous metadata leaks. Core *Information Technology* (IT) security implementations used these metadata leaks to perform necessary evaluations of client network traffic. Encryption of metadata such as *Server Name Indication* (SNI), *Application-Layer Protocol Negotiation (ALPN)* and more, requires a new outlook for how to safely adapt to current day privacy improvements. Authors present current day issues observed with ECH, a general outlook on protocol improvement areas and how to adapt to new challenges ahead in organisational environments. IT-security solutions are presented for their intended ability of securing IT-infrastructure and evaluated if they can continue operational functionality in relation to ECH. Authors present a simple and open source solution for content filtering, and examining the solution in various environments. The solution is deemed to be primarily applicable for organisations as supporting implementations are required for the solution to be deemed as effective. The report addresses concerns issued by researchers on how to safely incorporate *DNS over HTTPS* (DoH) and ECH-enabled websites to foster the adoption process.

Keywords: Encrypted Client Hello, ECH, Encrypted Server Name Indication, ESNI, TLS 1.3, Privacy, Privacy Enhancements, Security Implementations, Content Blocking, TLS Extensions, RPZ, Response Policy Zone, DNS Filtering.

Contents

1 Introduction 1
- 1.1 Aims 2
- 1.2 Report Outline 2

2 Background 3
- 2.1 Preliminaries / Concepts 5
  - 2.1.1 Content Delivery Networks 5
  - 2.1.2 Transmission Control Protocol/Internet Protocol Model 5
  - 2.1.3 Risk Acceptance Criteria 6
- 2.2 Encrypted Client Hello Research Background 6

3 Problem Formulation 9
- 3.1 Study Aims 10
- 3.2 Delimitations 11

4 Methodology 12
- 4.1 Hypothesis 13
- 4.2 Data Collection 14
- 4.3 Variable Selection 14
- 4.4 Data Analysis 14
- 4.5 Validity and Reliability 15
- 4.6 Ethical Aspects 16

5 Implementation 17
- 5.1 Implementation of Laboratory Environment 19
- 5.2 Implementation of Solution 24

6 Results 26

7 Discussion 31
- 7.1 In Relation to Previous Research 34
- 7.2 Ethical and Societal Aspects 35

8 Conclusion 37
- 8.1 Future Work 37

References 39

Appendices 43
- Appendix A - named.conf.options 43
- Appendix B - named.conf.local 44
- Appendix C - Slave Config for named.conf.local 45
- Appendix D - Master db.name_to_IP 46
- Appendix E - Master Reverse db.name_to_IP 47
- Appendix F - OpenSSL Installation Guide 48
- Appendix F.1 - OpenSSL Installation Troubleshoot 49
- Appendix G - nginx Installation Guide 50
- Appendix H - Completing ACME Challenge 51
- Appendix I - Permission Changes for Accessing SSL Certificates 52
- Appendix P - nginx.conf 59
- Appendix Q - DoH Client Configuration 61

Appendix R - ECH enabled-Website Example 62
Appendix S - RPZ Configuration of named.conf.options 63
Appendix T - Step-by-Step Guide on Obtaining “Encrypted ClientHello” Flag 65
Appendix U - Division of Work 67
Appendix V - TLS ECH Handshakes in Wireshark 68

1 Introduction

Encrypted Client Hello, or ECH for short, is a privacy enhancing extension for TLS version 1.3. TLS is a cryptographic communication protocol used in secure network communication. If implemented correctly, TLS offers considerable privacy enhancements and ensures data privacy, confidentiality and integrity proceeding the exchange of vital information. TLS is used in various types of network communication, ranging from web browsing, email, *Voice over IP* (VoIP) and more. The primary purpose of TLS is to ensure that data sent between two communicating parties, cannot be read by external endpoints. Over the years, TLS has been subjected to numerous exploits (Li et al., 2021; Merget et al., 2021), infringing on the intended functionality of the protocol. This has been dealt with constant revisions, adding new features and patching documented vulnerabilities.

TLS has gone through many iterations over the years with the most recent version being 1.3. The primary improvements lay in the so-called TLS "handshake", which is a special way of establishing a secure connection. The handshake involves agreeing upon certain criteria and exchanging information such as cryptographic keys in addition to other parameters. The TLS handshake is the backbone of establishing a secure connection where each step is significant. The update between TLS version 1.2 and 1.3 mainly solved previous exploits, the support of insecure cryptographic algorithms were phased out in addition to handshake revisions (Dierks & Rescorla, 2008; Rescorla, 2018).

With TLS only guaranteeing data privacy, confidentiality and integrity post the exchange of vital information, this leaves valuable and identifying attributes available for prying eyes prior to the exchange. Third parties such as *Internet Service Providers* (ISPs), government agencies, cybercriminals and other types of potential eavesdroppers exploit this fact. ECH intends to solve this issue, Encrypted Client Hello, as the name states, partially encrypts data fields which were subjected to traffic sniffing. Due to many *Information Technology* (IT) security solutions relying upon these data fields (Hamilton et al., 2020), encrypting them can cause unforeseen consequences for organisational networks.

For end-users, ECH is an extension which provides added privacy, but there is one primary issue. Partial information on what websites clients are visiting is no longer available for potential eavesdroppers at the data application layer (Rescorla et al., 2024). This data, among other uses, is also employed by IT-security solutions for web content moderation. With the addition of ECH, some IT-security solutions can observe a reduction in functionality (Rescorla et al., 2024), this is rather problematic for organisations which may require stricter network and system security. If the aforementioned problem is not addressed, the added ECH privacy enhancements may cause IT-security gaps in the local network.

1.1 Aims

The aim of the study is to find a solution on how to govern which ECH-enabled websites should be accessible, without involving the responsibility of CDNs. ECH is a user privacy focused TLS 1.3 extension which as of writing time, is in early phases of *Request For Comment* (RFC) publication. There have been notable concerns regarding pirating websites using ECH for their benefit as to avoid regional content restrictions (Van der Sar, 2023). The general consensus is that the primary responsibility of what content is to be accessible lies in the hands of CDNs. System administrators not being able to filter network traffic on the internet or application layer, which many IT-security solutions are based on, might come off as a challenge. This report will present a low-cost and open source solution on how to permit usage of ECH in an organisational network, whilst moderating which websites clients can access.

1.2 Report Outline

The research paper is aimed towards the computer science research community, security implementation specialists and individuals who are following the development of the ECH. Chapter 2 represents the background of ECH, describing dependent concepts of ECH as well as research background about ECH. Chapter 3 formulates the problem formulation regarding study aims as well as delimitations for this research paper. Chapter 4 covers methodology for this research, as well as validity threats, variables, hypotheses and how the research will be conducted. The implementation chapter or chapter 5, describes how the laboratory environment and solution was implemented. Chapter 6 shows the results of the implementation. The discussion chapter represents a discussion of results, and what *United Nations* (UN) *Sustainable Development Goals* (SDGs) are covered in the thesis. Chapter 8 describes conclusions made based on the results as well as future work. In the end of the research paper are references as well as appendices with configuration files and step-by-step guides.

2 Background

Since personal information started gathering on the internet, new challenges were introduced for both individuals and for organisations handling such information. Mitigations for encrypting information over the internet were introduced by the incorporation of *Secure Socket Layer* (SSL). SSL is an encryption based protocol, developed in 1995 by Netscape for its purpose to ensure authentication, integrity and privacy within internet communication (Kraus et al., 2020). SSL relies on a special handshake for ensuring data confidentiality. SSL usage was easily defined by a secure connection, which can be identified by *Hypertext Transfer Protocol Secure* (HTTPS) (Kraus et al., 2020).

In the year of 1999, a new version of SSL was being developed, which in turn evolved into the successor TLS. Both these protocols have one common factor, when establishing SSL or TLS communication, special handshakes are used for establishment. In the case of TLS, the handshake consists of a couple parts: The first part is the three-way TCP handshake, the second part consists of messages exchanging vital information such as ClientHello, ServerHello, Client Key Exchange and agreeing upon cipher suites (Göppert et al., 2023). The handshake varies depending on the TLS version where for each version, security and efficiency improvements can be observed. When a client visits a website, the initial steps involve a SYN message sent to the web server, then the web server responds with SYN ACK message to the client, lastly the client sends an ACK message. TCP three-way handshake is to confirm that both parties are ready to communicate with each other and that both parties can agree upon sequence numbers. Proceeding the TCP three-way handshake a ClientHello message defines the beginning of the TLS initiation process, where the server acknowledges the ClientHello message and sends a ServerHello.

To understand ECH as a TLS extension, it is necessary to explain what SNI does and its capabilities. SNI as defined in the RFC 6066, TLS in itself, does not have a mechanism to give out SNI information on which server is requested by the client (Eastlake, 2011). A server that receives the ClientHello message including SNI, may use that information to provide an adequate certificate to the client in order to establish a TLS connection. Being able to manage which certificate is to be presented, allows for verification that correct service is being communicated with. In other words, SNI resolved a problem where one public *Internet Protocol* (IP) address was pointing to only one server, but thanks to SNI, multiple web services can be pointed to a singular public IP address, identifying each server with the SNI attribute which the client obtained from DNS server response.

As SNI is natively unencrypted, the data field is readable when subjected to traffic sniffing due to the exchange of SNI information occurring prior to the key exchange (Koshy et al., 2023). SNI information being unencrypted was initially resolved by *Encrypted SNI* (ESNI), functionality was provided to encrypt SNI so that only the server and the client can decrypt the data field. During the TLS handshake, ESNI encrypts only the SNI data field in the ClientHello message, meaning that there is a layer of data protection. However, at the time ESNI was implemented, the remaining ClientHello message was still sent in unencrypted format, meaning that ClientHello was still subject to traffic sniffing. Since ESNI was not a complete solution for encrypting valuable information in the TLS 1.3 handshake, an idea was proposed for encrypting the ClientHello message, partially. ESNI eventually evolved into ECH in the same datatracker and is a successor to ESNI.

ECH was established on June 1st 2020 as a draft. ECH is based on two dependencies: The first one is DoH, and the second one is the TLS 1.3 extension ESNI (Rescorla et al., 2024). DoH's primary function is to resolve domain name to IP address DNS queries with the addition of encrypting DNS queries and responses, meaning that only the DoH server and the client can decrypt DNS server-client communication. DNS queries are natively done on port 53, while DoH is done on port 443, utilising the HTTPS protocol. DoH was firstly introduced in October 18th 2017 as a draft, today known for the RFC standard 8484 (Hoffman & McManus, 2018).

To understand ECH, a more thorough explanation of specifics is required. ECH splits the ClientHello message in two, the first message named outer ClientHello is an unencrypted message containing key shares and outer SNI, destined for intermediary servers. No valuable information is possible to extract due to target server SNI not being contained in the outer shell. The second message named inner ClientHello contains target server SNI which was previously subjected to traffic sniffing. Inner ClientHello encrypts the SNI field by obtaining target server public key through DoH, removing the ability of traffic sniffing on the SNI data field and DNS queries. Without obtaining the target server's public key, the ECH employed key pair encryption method would not be feasible (Rescorla et al., 2024). ECH also encrypts additional data fields in the inner ClientHello which were used for identifying traffic, for example ALPN and key shares (Rescorla et al., 2024).

TLS extension ALPN, has a feature to negotiate which protocol is to be handled over a secure connection, resulting in improved efficiency and avoiding additional round trips (Zirngibl et al., 2023). In other words, a client and a server make an agreement on which protocol both client and server supports and use the agreed upon protocol to communicate. ALPN can be a good indicator of what type of service a client is communicating with, thus requiring encryption, something ECH incorporated which ESNI lacked.

When it comes to ECH support on client web browsers, currently ECH is supported on the following browsers: Mozilla Firefox as of version 118 and onwards, Google Chrome as of version 117 and onwards, Microsoft Edge as of version 105 and onwards. Web browsers based on aforementioned web browsers may support ECH, verification is required for each specific web browser. As ECH adoption develops, more and more browsers will implement ECH support, however, since ECH does not have an established RFC standard, this extension remains under development.

ECH plays a role in Network- and system administration since this is a step forward for the network security, privacy and data integrity. The TLS handshake will be partially encrypted, increasing user privacy due to ECH development. For network administrators together with system administrators, there is a need to collaborate to achieve a new security feature for web servers, correspondent nameservers of the domain or the entire IT-infrastructure as ECH introduces new potential issues and changes to common services incorporating TLS 1.3.

2.1 Preliminaries / Concepts

The subchapter describes commonly mentioned concepts in more detail, understanding definitions and concept functionality will aid in the process of grasping the entirety of the thesis.

2.1.1 Content Delivery Networks

CDNs are distributed systems which deliver web content across the globe, offering their services to a wide variety of customers. The primary issues CDNs solve is the efforts required to have web services accessible globally with moderate to low delay (Wong et al., 2013). Whether streaming audio or video, requesting files or web pages, CDNs handle this operation by saving each servers' content across the globe ready to send the data to each requesting client. CDNs offer easy upwards scalable solutions with relatively low cost and are used by everyone from small corporations, huge conglomerates and individuals.

Content provided by CDNs is hosted on primary servers which distribute to so-called "edge servers". These edge servers are located in various geographical locations, when content is first requested, the client first comes in contact with a proxy server closest to the requester's geographical location (Wong et al., 2013). The proxy server who just received this request forwards it to an edge server closest to the clients geographical location. In turn the edge server requests this information from a primary server, also known as origin server, the origin server complies with the request, sending the data to edge servers which later caches the information for the next time someone requests the same data. The process of caching content on edge servers implies that there is no need to retrieve the original content from the origin server depending on cache *Time To Live* (TTL), thus saving bandwidth and reducing response times. Proceeding the retrieval from the primary servers or from the caching process, the edge server deals with the clients request, delivering the content to CDN proxy which in turn delivers it to the client (Wong et al., 2013).

Moreover, CDNs offer various types of cybersecurity mitigations such as never exposing the primary server's public IP address and closely monitoring traffic incoming and outgoing from proxy servers (Guo et al., 2018).

2.1.2 Transmission Control Protocol/Internet Protocol Model

The TCP/IP model is a conceptual framework that includes four layers consisting of network access, internet, transport and application layer (Alani, 2014). The aforementioned layers are vital to ensure that communication over the network is functional and if one layer is failing or is misconfigured, it will reflect on the whole communication process.

- **Network Access Layer**, known as the data link layer, handles IT-infrastructure such as ethernet cables, switches, routers and network interface cards.
- **Internet Layer**, known as the network layer, primary function is to ensure that packets go from source to destination making use of the *Internet Protocol* (IP) to forward packets to the next step.
- **Transport Layer** facilitates communication between two parties allowing for reliable transmission with the use of TCP/UDP/SCTP and other protocols. Each protocol varies in strengths and weaknesses and is applicable for different types of communication requirements.

- **Application Layer**, which most users interact with on a daily basis, is classified as a group of applications which lets users interact with the network. These applications can be everything from emailing clients to messaging apps or other software which require network connectivity.

2.1.3 Risk Acceptance Criteria

Risk acceptance criteria defines an organisation's accepted level of risk according to organisation's policies, objectives and interests of stakeholders. Since risk is impossible to eliminate, risk acceptance criteria is required to be developed so the organisation's management can evaluate current risks and measure if they are within desired levels (Svenska Institutet för Standarder, 2022).

2.2 Encrypted Client Hello Research Background

With advances in encryption and a greater populus consciousness about privacy, built-in metadata leaks in the TLS protocol are under scrutiny. The initial phases of establishing a TLS connection consists of ClientHello and ServerHello, key identifying information can be found here such as SNI, ALPN and more, ESNI/ECH intends to solve this issue. Privacy gain in ECH is a challenge to academically measure as of now due to no widespread adoption. As ECH incorporates more metadata under encryption than its predecessor ESNI, limited correlations can be theoretically evaluated. Until extensive research is possible, affirmative answers cannot be obtained as the real world impact is not currently accurately measurable.

TLS implementations consist of 0.06% for version 1.2, respectively 98.41% for version 1.3 according to data obtained from websites in Alexa Top 1M (Chai et al., 2019). This shows that 98.41% of Alexa Top 1M websites would be compatible with ECH. Present data on ECH implementation is hard to obtain as ECH is under development, currently ECH is disabled at the only CDN which allowed for simple implementation of ECH, previous numbers show that 92.56% of websites behind Cloudflare supported ESNI as of 2019, current date support of ESNI is unclear (Chai et al., 2019). Since deployment of ECH rolled out on all tiers of Cloudflare customers, likewise numbers can be expected as the primary criteria for ECH compatibility with web servers is TLS version 1.3. ESNI/ECH have the main core functionality, encrypting SNI and forcing DoH, ECH differs in incorporating more metadata under encryption. On a general outlook, eavesdroppers have a harder time identifying network traffic as only source and destination IP addresses are of value (Chai et al., 2019). Regarding identification of network traffic, also known as *Traffic Classification* (TC), ECH creates issues in relation to correctly classifying traffic for *Quality of Service* (QoS) applications (D. Shamsimukhametov et al., 2022). With potential privacy improvements comes drawbacks, for example, mobile operators use SNI to classify network traffic of services which do not count for clients purchased data plans. With the addition of ECH, this would become a giant hurdle as the SNI field would not be inspectable by third parties (D. Shamsimukhametov et al., 2022; D. R. Shamsimukhametov et al., 2023).

Trevisan et al. (2020, 2023) voice their strong concerns against DoH not being sufficient in protecting users DNS queries, if sufficient amounts of observable DNS plain-text data can be obtained, with the help of machine learning, predictive conclusions can be taken on DoH traffic. Trevisan et al. (2023) have suggested that predictions with up to 80%

certainty can be achieved by machine learning models. Penetrating the data confidentiality of DoH would lead to ESNI/ECH being rendered as useless as SNI is obtained from DNS queries. The client requested DNS queries would be predictable with approximately 80% certainty, this is not a perfect prediction percentage but it does not need to be as mass scale passive eavesdropping could be effective.

Protection of client DNS traffic relies on DoH. DoH is based on DNS over HTTPS and *Domain Name System Secure Extension* (DNSSEC). DNSSEC is designed for signing a DNS zone and validating answers that are provided through the DNS recursion. Thanks to DNSSEC, cache poisoning, domain hijacking, and network hijacking can be prevented (Chai et al., 2019). When combined together, DNSSEC mitigates cache poisoning, domain hijacking and network hijacking, while DoH ensures encryption of DNS queries so that passive network surveillance cannot see DoH traffic, DNSSEC ensures also that the DNS responses are valid.

Current day content filtering revolves around three primary fields of data; IP addresses, DNS responses and SNI (Chai et al., 2019). While the most prevalent filtering method by large margins remains to be IP filtering, as web services moves to more decentralised co-hosting solutions or CDNs, this might be subject to change due to the scale of collateral damage (Chai et al., 2019; Satija & Chatterjee, 2021). This observation is made by analysing trends whereas more and more websites utilise some sort of co-hosting solution. Shared hosting is expected to grow by 10.3% annually, while growth in a market does not directly correlate to the amount of unique IPs per website, it gives an indication of market trends whereas dedicated hosting is only expected to grow by 5.6% (Schäferhoff, n.d.). As of now, very mild collateral damage regarding IP blocking is observed within China. If more and more websites use co-hosting solutions, the amount of unique IP addresses will be reduced and the chance of collateral damage is increased. Collateral damage is specified to be web services which have been affected by IP blocking efforts unintentionally, when other web services hosted behind the same IP addresses have been targeted. This happens due to multiple web services being hosted on the same IP address, for example, if IP address 1.1.1.1 hosts x.com, y.com and z.com, if x.com is to be targeted by IP filtering efforts, all web services will be affected which are located on the IP address 1.1.1.1. Why content filtering efforts are of interest is due to the added ability of tracking users based on which web services clients are visiting, in turn, violating their privacy (Satija & Chatterjee, 2021). If metadata found in client-server communication is used for content filtering, it is also possible to use the same metadata for eavesdropping and profiling en masse.

An important note to keep in mind is the dataset which Chai et al. (2019) used to come to this conclusion, Alexa top 1M ranks the world's most popular websites and this was used as a dataset to obtain a list of websites to check for potential content restrictions put in place. ISPs can easily implement IP filtering methods to block undesirable content, but in the current contemporary landscape of the internet, this comes with a cost (Satija & Chatterjee, 2021). As previously mentioned, collateral damage in IP filtering efforts are evaluated differently by each country. Countries vary in their approach to content filtering, for example, China favours IP filtering if collateral damage is within unknown acceptance levels (Satija & Chatterjee, 2021). An important note is that China employs a mixture of filtering methodologies with the most prevalent being forementioned. India on the other hand favours DNS and HTTP filters, with most recently employing SNI filtering (Yadav et al., 2018). Effectiveness of circumvention of content censorship may vary depending on country filtering efforts. Just because content filtering efforts are not implemented, does not imply no mass eavesdropping or profiling is occurring.

Satija & Chatterjee (2021) voice their strong recommendations in a more rapid adoption of ESNI/ECH as benefits in privacy and regional censorship circumvention could be found. Suggestions are based on factors relating to the cost of blocking legitimate traffic if ESNI/ECH was to be deemed as a threat. If ECH is only used by clients accessing what is deemed as questionable content, or the amount of clients using ECH deemed as being low, blocking the usage of ECH would be of little detriment and would be a rapid decision. Contrary if ECH adoption subsequent to official RFC publication rapidly occurs, this decision would be much more difficult as legitimate traffic would get affected.

In modern day, internet services are dynamically adapting to diverse network situations to provide the best quality and end-user experience. As network conditions change, frame-sizes, data rates and other factors dynamically adjust to provide the best quality and end-user experience. Client-server communication can occur on many different channels, beside primary channels which carry main service payload for the website, additional channels such as side-channels can be used for transferring intermittent information such as ad-insertions (Khandkar et al., 2023). When implementing an ECH solution on a website, there is a privacy concern when side-channels are being used. Side-channels mainly use TLS version 1.2, due to low overhead requirements of the initial handshake compared to TLS 1.3, making it a more favourable option for transport of non-important data (Khandkar et al., 2023). However privacy concerns are present due to the usage of TLS 1.2 which leaks metadata that can be accredited to primary channels which may be under ECH. Due to fundamental differences in the TLS handshake between the versions, no ECH implementation is possible for TLS 1.2 unless major restructuring is done. TLS 1.2 natively exposes parts of communication such as server certificate and SNI in plaintext, where later it can be tied to the primary channel and main client-server communication.

Chai et al. (2019) voice their strong recommendations for communication using TLS 1.3 with the addition of ECH to not fallback to TLS 1.2 or to non-ECH-enabled communication if privacy is to be upheld.

As ECH's primary goal is to enhance privacy in TLS version 1.3, an important factor is to further remedy previous metadata leaks such as ALPN. ESNI had flaws in not fully protecting the user as only SNI was encrypted, ECH intends to encrypt key identifying data fields in the entire ClientHello message, such as ALPN. Core functionality of ALPN is for applications to specify which protocol should be in use during communication, this can be used as a part in a dataset for eavesdroppers to further categorise traffic (Friedl et al., 2014). If SMTP is specified in the ALPN data field, this would suggest that the client most likely is communicating with a mail server. With the addition of ECH, the ALPN data field is encrypted and can no longer be accessed by prying eyes.

With relative privacy gains for end-users using ECH where less of their metadata is available for eavesdroppers, are there any entities which may benefit from these circumstances? ISPs are still to this day served with tons of metadata such as SNI, ALPN and unencrypted server certificates as ECH is not well established yet. If ECH is to be accepted en masse, giant hurdles such as side-channel leaks, DoH predictive attacks and uncertainties if countries will allow ECH all together, need to be thwarted. One could ask which responsibilities CDNs gain in such an environment as governments may pressure for regulatory control. Further research is required once ECH establishment is underway as only speculation is currently possible.

3 Problem Formulation

There have been notable concerns issued with prohibited sites circumventing regional and local content blocking efforts with the usage of TLS 1.3 extensions ECH (Van der Sar, 2023). Due to the fact that ECH is relatively new, there is a knowledge gap in how to safely permit ECH in organisational environments.

There are multiple ways to create a safe and secure IT-environment in organisational networks, one of them is implementing firewalls. Firewalls can be implemented in many different ways, such as on a router or a specialised hardware appliance. The primary functionality of a firewall is to inspect incoming and or outgoing packets, comparing them to specified rules set on what is to be let through and what is to be blocked. Firewalls act like barriers between an internal trusted network and an external untrusted network.

Firewalls can be configured to inspect various information to make acceptance or rejection decisions, such as source and destination IP addresses with port numbers. There are lists of known malicious IP addresses which have been suggested to provide questionable content or perform malicious actions against clients and servers. This is all good in theory until CDNs and proxies come into the picture.

As per previous definition of CDNs in subchapter 2.1.1 Content Delivery Networks, functionality is introduced where the corresponding destination IP address on the internet layer does not relate to the requested web server. To identify which server the packet is destined to, SNI is used which allows multiple hostnames to be served on the same IP address. There is no correlation between legitimate CDNs providing their services to malicious actors but there are documented cases of various forms of intrusion, creating a need for network- and system administrators alike to manage connections incoming or outgoing to CDNs. One of the most popular CDNs did have a vulnerability where remote code execution was possible, compromising website content via the popular CDN (Sharma, 2021).

Deep Packet Inspection (DPI) firewalls, also known as a packet sniffing firewall or a type of middlebox, examine a vast range of metadata in addition to IP addresses and port numbers (Hamilton et al., 2020; Yang et al., 2010). DPI firewalls would be suitable for making decisions on allowing or rejecting the incoming or outgoing network traffic if SNI is of interest. ECH creates an issue where the functionality of a DPI firewall is limited due to only the outer ClientHello SNI being visible, not containing the target server's SNI. This leads to troubles identifying what server is being accessed on the application and internet layer, eliminating partial DPI firewall functionality. For such a solution to be effective, *Man-in-the-Middle* (MitM) attacks need to be performed on a trusted network, requiring expensive hardware which relies on complex operations, increasing network latency and or reducing speed (Hamilton et al., 2020).

Cloudflare provides CDN services and is estimated to host over 12.7% of the internet (Sharma, 2021). Upon recently, Cloudflare had enabled ECH functionality for all customers (Cloudflare, 2023), creating a potential issue. When ECH is enabled on the CDN level, how is it possible to ensure that content requested from said CDN, is something to allow on the local network. It is not practical to block Cloudflare's public IP address range, as that leads to substantial parts of the internet not being accessible. DPI firewalls inspecting the data on the application layer are also of little use as the inner SNI is encrypted under ECH, thus this solution will not work either, if ECH is to be allowed on the network. DPI firewalls with MitM attack functionality, may be used for such applications. An issue with DPI firewalls is that

they perform costly operations which reduces network performance substantially (Hamilton et al., 2020). A solution would be to disallow ECH on the network all together, but there are privacy gains to be obtained if ECH was to be allowed. So what is the solution if decisions cannot be made on the internet nor application layer in organisational networks?

3.1 Study Aims

This study aims to reduce the current knowledge gap in ECH functionality and provide documentation on how to manage ECH in an organisational environment. The study presents ECH functionality, providing potential issues which may appear if ECH adoption is to rapidly develop. Research is focused on managing access to websites with ECH functionality without involving actions taken by CDNs, which currently are seen to have the full responsibility of content management. As of recent, concerns have been issued where illegitimate websites using ECH to circumvent content restriction efforts with success (Van der Sar, 2023). Network- and system administrators need to be up to date to the constantly ever changing landscape of IT, new threats and vulnerabilities are created which can cause consequences outside of risk acceptance criteria (Svenska Institutet för Standarder, 2022), this is due to protocols evolving to solve current needs and problems and may change a lot of aspects within IT.

As mentioned before, this study aims to achieve two goals and thereby reduce those knowledge gaps. The primary aim is to provide content blocking solutions against ECH capable websites. ECH introduces new challenges for content moderation where less information is obtainable on what websites clients are visiting. To understand the concept of the solution, and draw a valid conclusion, somewhat deep knowledge about TLS and ECH functionality is required.

A sub-goal is to shed light on ECH as a whole, reducing knowledge gaps so IT-enthusiasts, network- and system administrators can get a better grasp of implementation examples. Many of the research papers and documentations do not describe to full extent on what is required for building an ECH capable website or are misleading. The thesis aims to clarify the prerequisites for building an ECH capable website as well as describing a simple configuration.

For widespread adoption of ECH, a method for filtering and blocking ECH websites is required. In order to redress this challenge, this research aims at providing a solution for blocking content based on ECH capable websites with DNS filtering.

ECH is currently in the early phases of adoption, if widespread adoption is to be expected, updates and revisions of current IT-security solutions are required. IT-security solutions such as firewall appliances are usually seen as set and forget, creating threats over time which may be outside of risk acceptance criteria (Svenska Institutet för Standarder, 2022). This report will shed light on the TLS 1.3 extension ECH, giving a better understanding of how to safely permit usage in organisational networks.

3.2 Delimitations

The research is conducted with the following delimitations which are specified in relation to ECH and can be seen as improvement areas for future or extended research.

- **Expertise Limitations:** The authors' expertise does not lie in software development nor cryptography, the study refrains from delving into modifying existing specialised security appliances such as stateful DPI firewalls.
- **Technology Limitations:** Middleboxes such as stateful DPI firewalls which can deal with simple HTTPS packet inspection, may be used to intercept and inspect what query is made between the client and DoH server, making decisions based on DoH response or client query. Such solutions are perceived as complex and expensive which would reduce scalability. Authors recognize that such a solution is feasible but the cost of inspecting each HTTPS packet is perceived as costly monetarily and inefficient in comparison to DNS filtering.
- **CDN Responsibility In Relation To ECH:** The shifted responsibilities of CDNs in relation to ECH is an open question which requires additional research and is difficult to quantify as of now, this is due to limited obtainable real world information, thus this question is excluded as of now.
- **DNS Filtering Limitations:** DNS filtering which the solution is based on, is prone to circumvention by proxies, *Virtual Private Networks* (VPNs) and more. Circumvention of DNS filters is a whole new subject which is well known and documented on how to thwart, thus only lightly mentioned in relation to implications.
- **Time Constraints:** Time constraints makes it difficult to realise the real world impact of IP filtering in relation to CDNs. Throughout this research, IP filtering methods for content blocking will not be the focus for this research.

4 Methodology

The laboratory setup required to verify the hypothesis consists of ECH-enabled websites and a client with an ECH capable web browser; this client should not be able to access the websites via web browsers once the implementation is active. In theory, ECH is possible to implement, however such research has not been done extensively in addition to explaining the TLS 1.3 extension ECH in detail, whilst documenting the implementation solutions.

The method chosen for this research paper was conducting an implementation, based on making a working TLS 1.3 ECH enabled website and being able to selectively filter the known ECH websites. The reason why implementation activity was chosen as a primary research method was due to lacking previous academic implementations and papers within the subject. As of publication time, there are not enough papers within the subject to conduct any form of literature study to find valid IT-security implementations, this is most likely due to ECH not being widely adopted. Additionally, ECH is lacking a formal RFC publication as development is still underway, making it difficult to base a study on a document subject to change. Surveying organisations about current IT-security implementations, verifying if they would be sufficient for handling ECH and finding the best solution, could be viable although difficult. Due to a select few web services supporting ECH, it might not be relevant and no thoughts have been given for it. Adding to the difficulty of surveying organisations about their security solutions, would be procuring a large enough sample size as there is a lot of secrecy within this subject. Not only would the sample size be an issue, sharing such detailed information about a large sample size of organisations security implementations would be an ethical concern, as malicious actors might make use of this information. Conducting an interview with IT-security professionals, knowledgeable within ECH, where ideas or already complete implementations for how to securely handle ECH in corporate environments, would be subject to validity threats as the sample size possible to procure for the given time period of conducting this study would be limited.

To prove that it is possible to implement ECH in the given environment, with the requirement of blocking access to specified websites, there is a need to firstly recreate such configuration in a laboratory environment which could be used for future reference. The purpose of the laboratory activity was to document the whole process so that IT-competent personnel can recreate the ECH implementation and verify the given solution with this research paper. The filtering solution can then later be applied in a live environment proceeding the verification process.

There are multiple ECH implementations currently available, mostly undergoing development, this laboratory setup will be one of the simpler ones, not including CDN functionality, rotating keys nor proxy servers. Prerequisites for the implementation is creating an ECH capable website which will be tested against, is to have two DNS servers. The primary DNS server acts as a master DNS server with DoH capabilities and an additional slave DNS will be present. The need for two DNS servers bases itself in requirements from domain registrars, which usually require a backup DNS as if one fails or to provide load balancing. ECH requires DoH capabilities from client queried DNS servers to not allow for traffic sniffing. Only official stable versions of applications are used in the implementation of DNS solutions.

The ECH enabled web server requires dependent libraries available for download from specified GitHub repositories. Currently, OpenSSL and/or nginx does not support ECH on stable versions and is only available from unofficial releases. This may be subject to change, verify current version capabilities before attempting to replicate installation procedures provided in implementation section 5.1. Hyperlinks are provided to GitHub repositories for said unofficial releases.

Web server system application requirements.

- OpenSSL
- nginx
- Domain
- Let's Encrypt CertBot
- SSH

DNS server application requirements.

- BIND9
- Domain
- Let's Encrypt CertBot
- SSH
- nginx

As for ECH content blocking, an additional DNS server is required. The third DNS server acts as a recursive DNS, it forwards the received queries to the primary DNS server if local records cannot be obtained. In real world applications, the DNS filter would set its forwarder to a public popular DNS provider. The recursive DNS server requires DoH capabilities, as well as Response Policy Zones, shortly RPZ. The chapter 5.2 explains further on how RPZ has been configured.

To reiterate, each virtual machine made in VPS will have 1 virtual CPU, 2GB of RAM, and 55GB of SSD storage. The DNS server with DoH capabilities has the only difference of the *non-Volatile Memory Express* (nVME) storage.

4.1 Hypothesis

As part of the implementation and the research, two hypotheses have been formulated. H0 is presented as the null hypothesis while H1 is the alternative hypothesis. H1 will be used as the alternative if H0 is rejected. To determine the region of rejection, an alpha (α) level will be used, the common significance level of 0.05 will be employed. (PennState Eberly College of Science, n.d.-b)

H0: It is not possible to implement DNS filtering for moderating access to ECH-enabled websites

H1: It is possible to implement DNS filtering for moderating access to ECH-enabled websites

4.2 Data Collection

For gathering data, multiple web browsers will be tested for ECH functionality based on the laboratory ECH implementation on the website secret.ech-website.company. Each browser will be tested for ECH support, by visiting the website secret.ech-website.company and verify if ECH has been established or not. Browsers being tested for the ECH support and functionality are: Microsoft Edge, Mozilla Firefox, Google Chrome, Opera GX, Mozilla Firefox ESR and Brave. Firefox *Extended Support Release* (ESR) will be used as a baseline for non-ECH support, this is to validate that the solution can coincide within an environment that does not fully support ECH. Firefox ESR as of version 115.10.0, does not have implemented ECH support, and will be used for controlling backwards compatibility for ECH enabled websites, for example, whether or not TLS 1.3 connection will be established.

For each web browser, a DNS filter will be used, which is https://filter-website.company/dns-query. On this DNS filter, response policies will be modified to allow or restrain domain name resolution. After each test, results will be noted in binary form as true or false.

4.3 Variable Selection

The following effort is for describing the chosen variables for the implementation. Independent variables are the variables that can be altered in the verification of implementation solutions. Dependent variables can be determined based on the changes on independent variables.

For this study, the chosen main independent variable is the DNS filter RPZ configuration. The main dependent variable is website accessibility.

4.4 Data Analysis

In order to analyse the gathered data, Fisher's exact test will be used due to the small sample size of data, Fisher's exact test is perfectly suitable for data analysis of the 2x2 contingency table created by the variable selection (PennState Eberly College of Science, n.d.-a). The high probability of a perfect separation in the dataset excludes binary logistic regression, perfect separation is when the X independent variable directly influences the dependent Y variable. Due to only two variables being tested, and if the solution works as it expected, likelihood and other estimation functions of binary logistic regression will not be computable or return strange results. Fisher's exact test is suitable for small sample sizes and can deal with potential perfect separations unlike binary logistic regression. Additionally, the Chi-squared tests are not reliable with data containing zeroes, which can return unreliable results. Fisher's exact test is employed to observe whether a non-random statistically significant relationship between two variables in a contingency table can be observed, Fisher's exact test is sufficient for answering the null hypothesis as the primary objective is to observe if there is a statistically significant relationship between the independent and dependent variables.

4.5 Validity and Reliability

Conclusion validity: DNS filter RPZ in relation to websites' accessibility will be tested for a potential statistical significant relationship with Fisher's exact test, primarily by calculating the p-value observed between the independent and dependent variable. Many technical factors attribute to website accessibility being heavily influenced by configuration of DNS filter RPZ. Technical aspects strongly suggest that client made DNS queries are paramount if a TLS 1.3 ECH-enabled website connection is to be established. The primary reason for this claim is the requirement made from the client to resolve a domain name to a public facing IP address, without this capability, the client has no information on which IP address the target server is located on. Secondly, to perform key-pair encryption which is required for encrypting SNI, the client is first required to obtain the HTTPS *Resource Record* (RR) type from a DoH server. If the DNS filter does not provide resolution services for a specific domain, nor the public key, nor the SNI can be obtained. To reiterate, configuration in the DNS filter RPZ, permits or denies DNS resolution capabilities for specific domains contained in RPZ, depending on configured response. The main objective is to test the possibility of implementing DNS filtering for moderating access to ECH-enabled websites; no confidence interval nor odds ratio is required to be calculated as that is futile when rejecting or accepting the null hypothesis.

Internal validity: To ensure that changes made in DNS filter RPZ are directly influencing website accessibility, a number of control tests will be issued. First and foremost the specified websites were tested for accessibility before applying the DNS filter. This was done by setting domain controlling DNS server of the domain ech-website.company, as primary DNS server for the client. Proceeding this action, websites' ECH capabilities were noted. To ensure that potential configuration errors in DNS filter RPZ did not interfere with the results, control tests were issued so domains not included in the DNS filter RPZ configuration file were still accessible. Small configuration errors in DNS database files, such as RPZ or zone files, can cause server error responses, making all DNS queries return with NX domain or server error. This issue could interfere with the results and thus creating the requirement of control tests.

External validity: This study has external validity limited to delimitations presented in subchapter 3.2. Due to the narrow scope, proof-of-concept-like presentation and data analysis, this study can be applicable to current day problems for organisations with Windows 10 and 11 clients requiring content moderation of ECH/ESNI capable websites. Additional verification and testing is required for definitive conclusions, the current study cannot accredit DNS filtering as the best solution for web content moderation with ECH in mind for all organisations. Due to concerns raised for various forms of DNS filtering circumvention techniques, and the varying countermeasures required, additional extensive research is necessary before a definitive conclusion can be taken. This study addresses the possibility of DNS filtering as the primary method for content moderation against ECH-enabled websites, weighing advantages and disadvantages by critical analysis against IT-security controls used by organisations as of now. This study introduces potential flaws of ECH and how it interacts with current day IT-security solutions such as firewalls, DPI firewalls, middleboxes and more.

Reliability: ECH as of writing time does not have an official RFC publication, thus delaying any rapid adoption process, this in turn can affect the results reliability if functional and structural changes are subjected to ECH. Although changes can occur, SNI is the only way to commonly differentiate web services hosted behind the same public IP address. DNS resolution is not only required to obtain the SNI data field, but also to obtain which public IP address the client should set in the destination IP field header. If SNI would change in a way where data is obtained from a differentiating source and not from DNS servers, the domain name to IP address resolution would still be required. This may lead to reduced functionality but the results would be most likely the same unless the client already has cached the domain name to IP address resolution.

Due to SNI being obtained via DNS or DoH, the solution is applicable for all versions of TLS with both ESNI and ECH TLS extensions. As there is no guaranteeing what will change with ECH, solution verification is required once official ECH RFC publication is completed.

4.6 Ethical Aspects

When content moderation and content filtering is in question, ethical dilemmas are always prevalent during such discussions (Hamade, 2008). The internet thrives on freedom of information and freedom of speech, where user privacy is a core building block required to achieve the aforementioned qualities. The solution presented for moderating content en large, can be used by ISPs to enforce censorship decisions taken by governmental bodies if the users' browsers are not correctly set up. With *Dynamic Host Configuration Protocol* (DHCP), ISPs can enforce DNS servers to each client obtaining an IP address lease (Droms, 1997). If browsers are not set up correctly such as using the default DNS obtained by DHCP, this solution could be used for web content censorship. This is nothing new but such scenarios would not allow for ECH functionality due to the specific browser settings required for ECH. Most common browsers require maximum DNS security settings to be selected where only trusted DNS providers are easily choosable. If ECH is to be enabled on a client browser, DNS obtained from DHCP can be overwritten by manually selecting preferred DNS.

5 Implementation

The implementation section consists of two parts, one part laboratory setup and one part solution. Both parts are required to verify full functionality, the decision of creating the laboratory environment and not basing the verification phase with websites solely out of authors' control, was due to the benefit of having full control over experimental conditions and to gain a greater understanding in the surrounding factors of ECH. Conducting the implementation with own servers gives more precise results and full overview of influencing factors. Once verification is completed within the laboratory environment, additional verification is done with ECH-enabled websites out of authors' control, this is to obtain more data to analyse and potentially stress test the solution. Solution visualisation can be observed in Figure 1. Here the website has no corresponding entry in DNS filter RPZ, allowing the client to resolve the DNS query.

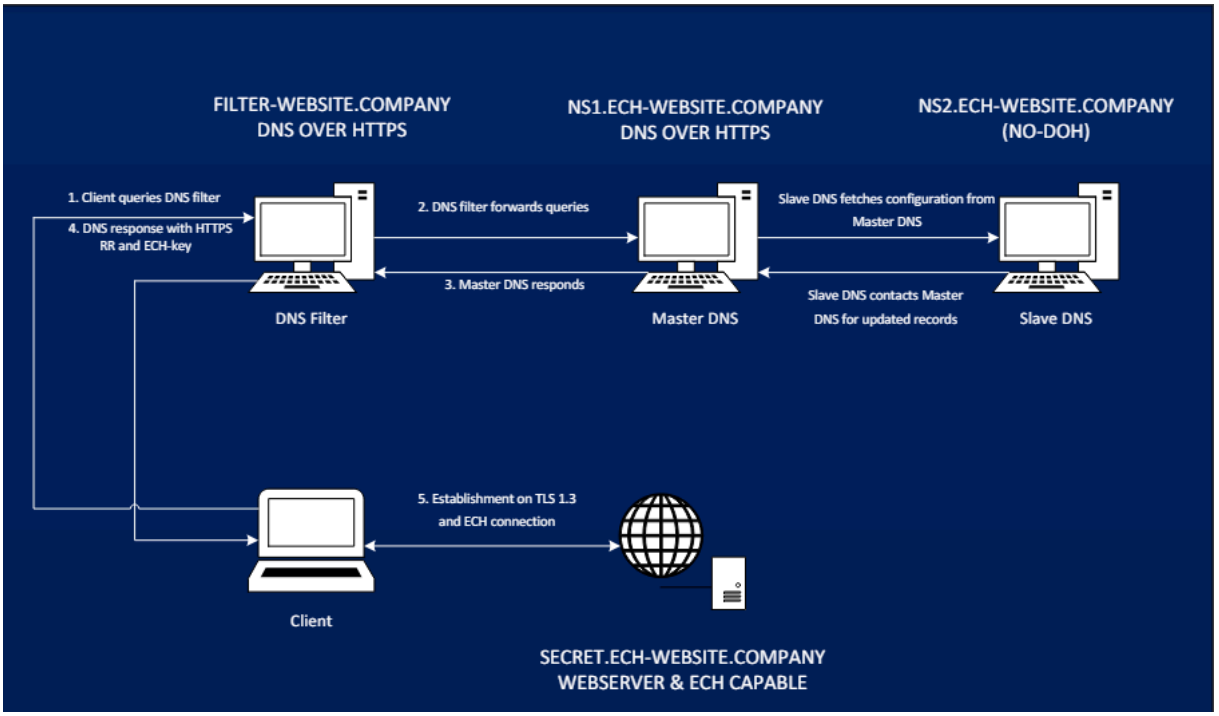


Figure 1. The laboratory solution topology if ECH website is permitted in RPZ

Figure 2 visualises when there is a corresponding entry in the DNS filter RPZ, this in turn blocks the client from resolving the DNS query where obtaining SNI, private key and resolving domain name to IP address is restricted.

Figure 2. The laboratory solution topology if ECH website is denied by RPZ

Five web browsers with ECH support were tested against the DNS filtering solution where the accessibility of ECH-enabled websites in and outside of authors' control was observed through changes in DNS filter RPZ. Firefox ESR in addition to other browsers, was used as the control browser and does not have native ECH support, this is necessary as to verify overarching functionality with outdated browsers. Laboratory functionality in Table 1 specifies if the ECH-enabled website created in following subchapter 5.1 implementation of laboratory environment, is accessible with full functionality without any content restriction efforts put in place. The DoH server used to conduct the initial tests is the domain controlling DNS server of domain ech-website.company, once initial tests were completed, the DNS filter was applied on all clients, the DNS filter was specified by the following *Uniform Resource locator* (URL) https://filter-website.company/dns-query. This is the same DNS server whose response policies are put in place to administer filtering efforts, not to be confused with the DNS servers responsible for the domain ech-website.company. See Table 1 for more information on each web browser and version used as well as whether ECH is supported and if the laboratory solution works as intended.

Browser	Version	ECH Support	Laboratory Functionality
Mozilla Firefox	125.0.3	True	True
Firefox ESR	115.10.0	False	False
Microsoft Edge	124.0.2478.67	True	True
Opera GX	109.0.5097.70	True	True
Brave	1.65.126	True	True
Chrome	124.0.6367.119	True	True

Table 1. Planning for data collection and testing the implemented ECH enabled website

5.1 Implementation of Laboratory Environment

Virtual private servers were used for hosting each virtual machine. The reason for choosing VPSs was due to the requirement for public IP addresses in order to complete Let's Encrypt *Automated Certificate Management Environments* (ACME) challenges, a key step in enabling DoH capabilities for the DNS servers and implementing the ECH-enabled website. Although self signed certificates can be used which would bypass the need for public IP addresses to conduct these tests, Firefox version 124.02 and most popular web browsers do not accept self signed certificates as a valid form of authentication. The implementation of an ECH-enabled website was inspired by Savin (2023).

Glue records for domain ech-website.company	dns.ech-website.company	70.34.221.99
	dns2.ech-website.company	70.34.201.228

Table 2. Domain Registrar Information

IPv4	**OS**	Software	Version
70.34.221.99	Debian 12	BIND9	9.18.24-1
		Certbot	2.1.0-4
		nginx	nginx-1.25.5 mainline

Table 3. DNS Server Information

Proceeding the purchase of a domain, it is important to start with DNS configuration for the two DNS servers. DNS propagation time for domain registrars may vary from one to 72 or more hours depending on registrar specifications, verify DNS changes directly to the affected DNS server.

Berkeley Internet Name Domain (BIND9) is an open source nameserver which fills all the criteria required such as DoH capabilities, zone control, DNS filtering and serving ECH public keys. BIND9 has the capability for serving DNS requests over the HTTPS

protocol, which is a requirement for establishing a TLS version 1.3 ECH-enabled connection. There are DNS resolvers such as dnsmasq, Pi-hole, dnsproxy, etcetera. BIND9 was chosen as a DNS resolver for this implementation, since authors were familiar with the configuration setup and fulfilled the aforementioned criterias.

To install BIND9 on both DNS servers, input: apt install bind9. BIND9 will be located at /etc/bind/, here all configurations will be made to each DNS server. Before any configuration is made, make sure to allow port 53 and 443 through the firewall as depending on VPS, these ports can be disabled by default, see Figure 3 for opening DNS and HTTPS ports.

```
ech@dns-server: sudo ufw allow 443
ech@dns-server: sudo ufw allow 53
```

Figure 3 . Port opening via Uncomplicated Firewall (UFW)

Once installation is complete, basic configuration for BIND9 functionality is required. Two primary files have to be configured before DNS records can be added, named.conf.options and named.conf.local (Debian Wiki, n.d.). Configuration file named.conf.options contains options for the DNS server, a forwarder will be set as this server will be tested against to verify all around functionality. Forwarder selection is at own discretion. However, for simplicity's sake and to eliminate any unforeseen issues in testing it is recommended to use any popular DNS provider such as Cloudflare DNS with the corresponding IPv4 address of 1.1.1.1, or Google DNS with corresponding IPv4 address of 8.8.8.8. For more details see Appendix A for the named.conf.options as the related configuration.

To verify eventual semantic errors use named-checkconf on configuration files beginning with named.conf. If no output is generated from the command there are no semantic errors present. Restart BIND9 and verify that no errors have occurred.

Once the initial configuration of named.conf.options is completed, zones have to be configured and related zone configuration files have to be created. Append content found in Appendix B in the named.conf.local zone configuration file, verify semantics with named-checkconf named.conf.local.

The named.conf.local file has to be edited on the slave DNS server but with changes made to specify role, as the slave DNS server will not contain any zone files locally and only retrieve them from the master DNS server. See Appendix C for the slave configuration of the DNS server.

The reverse zone, the first zone present in named.conf.local, varies depending on the IP address range of servers. Attribute in-addr.arpa is used to specify an IP address range for use in reverse IPv4 DNS queries. If a zone is to be made for the IP address range of 192.168.1.0/24, the zone would be specified by 1.168.192.in-addr.arpa.

Once zones are configured, create specified files which should contain the DNS records for each zone. This file needs to start with db.[**FileName**] for BIND9 to recognize them as DNS record databases. Copy over one of the premade templates to the file specified for each zone in the row “file”.

Once the creation of zone database files are completed, it is important to update the

serial number after each modification of the zone database files as to specify to BIND9 that the file has been updated. Best practice is to set the serial to the current date of modification and a serial number preceding the date as multiple changes may be made the same day. TTL is specified at the top of each zone database and can initially be reduced to a shorter duration as BIND9 caches heavily. If a record TTL has not expired but the said record is modified or removed, the record may still appear in queries which can result in confusion. Begin with adding and modifying the entries according to web server and primary DNS IP addresses. The name selected will be queryable with a proceeding dot and your domain. In this case, www.ech-website.company will point to the public IPv4 of 70.34.223.34, make sure this record points to the predetermined web server. Configuration of database files can be seen in Appendix D for forward lookup.

Just like in name to IP address resolution database, the reverse mapping works similarly. Given the reverse IP address zone, only the last octet needs to be used to identify the IP address. If the IP address range is on differentiating subnet, more octets are required to identify the IP address to the name or to create a new zone. Copy the premade template over to the specified file entry in named.conf.local for reverse lookup. Appendix E depicts modifications for the reverse lookup zone.

To recap the application list, chapter 4 showed what applications were needed for the web server, Table 4 shows how the web server was configured. Note that dependencies were the prerequisites in order to start with the installation of OpenSSL and the nginx. For the links of repositories, refer to chapter 4 Methodology.

IPv4	70.34.223.34
Software	OpenSSL
	Certbot
	nginx
Dependencies	build-essential libpcre2-dev zlib1g-dev libssl-dev
OS	Ubuntu 23.10.1

Table 4. ECH web server

On the web server, the software installed was OpenSSL and nginx from the previously mentioned GitHub repository, where both software have ECH support. The installed version of OpenSSL differs from the preinstalled one due to added ECH capabilities. This version of OpenSSL was used to create public and private ECH keys, in order to get the public key portion inserted into the DNS HTTPS RR, which was also required to benefit the ECH security implementation. The next paragraph shows in detail how a web server should be configured in order to utilise the ECH security implementation.

To begin with web server configuration, the software installations are stored under /home/ech/src. The prerequisite were to install following libraries and build tool in order to install OpenSSL and nginx: sudo apt install build-essential libpcre2-dev zlib1g-dev libssl-dev -y

Next was to download OpenSSL and nginx with the following commands: git clone https://github.com/sftcd/openssl.git and thereafter change to the branch by typing git checkout ECH-draft-13c. When downloading nginx, make sure to navigate back to src and type the following command to download nginx: git clone https://github.com/sftcd/nginx.git. Afterwards head to the nginx directory and type the following command to change to the right branch for the ECH support with the nginx: git checkout ECH-experimental.

When each download has been done it is important to open the ports in the firewall with port 80 and port 443 on the ECH-server and that is done by using Ubuntu's built-in *Uncomplicated Firewall* (UFW). By typing command ufw allow http && ufw allow https , the protocols HTTP and HTTPS are allowed in the firewall where ECH security implementation is being built on the HTTPS protocol. The next step was to install OpenSSL, which is described in Appendix F on how to install it. Appendix F.1 describes a troubleshooting process, in case installation goes wrong.

After the installation of OpenSSL is done, the OpenSSL will be installed under /usr/local/bin/openssl. It is required to use a shared library by typing the following command in order to use the development version of OpenSSL: sudo ldconfig /usr/local/lib64.

Next was to install nginx, and the installation process for nginx is described in Appendix G in detail, where the Appendix G goes into the installation and verification process.

After nginx has been deployed on the ECH server, next was to acquire SSL certificates from Let's Encrypt. Appendix H explains in detail on how certificates are being acquired. The same procedure for acquiring certificates is applied on the master DNS server.

Unless a wildcard certificate has been retrieved, the retrieval of certificates is to be repeated on the primary DNS server, as explained in Appendix H. To do this, nginx is required for proving ownership of subdomain.

Once certificates have been issued on the DNS server it is important to change permissions so BIND9 can access the files accordingly and the reading access according to Appendix I, which guides on how permissions should be configured for accessing the certificates.

Now that certificates for DoH are retrieved and correct permissions have been issued, DoH is to be configured. Configuration takes place by modifying content in /etc/bind/named.conf.options. Modify current version of named.conf.options to comply with the following updated version of the file, according to the Appendix J (Goldlust, 2021).

When SSL certificates have been acquired, it is important to import the SSL certificates and ECH key directory according to the nginx.conf configuration. For more details regarding the nginx.conf refer to Appendix P. The ECH directory where ECH-keys will be stored and created goes by the name of /opt/ech/conf/ech_keys.

In order to create ECH keys, type the following command: /usr/bin/local/openssl ech ech -public_name hidden.ech-website.company -pemout nameofthefile.ech.

Proceeding certificate retrieval and valid configuration, DNS records can be added so final verification can be completed. Appendix L explains further what DNS records are needed to be added for ECH to work.

Once the serial has been updated and BIND9 is restarted, it should be possible to visit the web server via the specified subdomain with an ECH-enabled web browser. To configure an ECH-enabled web browser Firefox will be used, navigate to the lines > Settings > Privacy and security > DNS over HTTPS should be set to "Maximum protection" with DoH server, in this case https://ns1.ech-website.company/dns-query. Afterwards it is important to check that the status for DoH is active, see the Appendix Q for an example. Navigate to the page with the intended inner SNI e.g. https://secret.ech-website.company. For more information, see Appendix R visualising how DoH configuration should look like from the client side.

To verify ECH connectivity there are two options, Wireshark and embedding HTML code. One option is to verify ClientHello packets with Wireshark and the other option is to observe content presented by the web server, indicating ECH connectivity. To add this content, edit /opt/ech/html/index.html with the following lines of code found in Appendix K. Appendix K further shows the index.html code implemented in the web server and the interpretation of the code.

When visiting the ECH-enabled web page, Appendix R prompt can be used for reference to verify if all components from the client-server side are correctly configured, where ECH could have been established with outer and inner SNI.

Another component required for testing the DNS filtering solution is the Windows 10 or 11 virtual machine. The filtering solution is explained in subchapter 5.2 on how the DNS filter was implemented. In this case Windows 11 version 23H2 virtual machine was installed in Hyper-V to test the different web browser support for ECH. The web browsers that were tested for ECH support were Google Chrome, Mozilla Firefox, Mozilla Firefox ESR, Brave, Opera GX and Microsoft Edge. ECH has a prerequisite to have DoH enabled, whether it is on the *Operating System* (OS) level or on the browser level. DoH will be set to the implemented DNS filter with a corresponding domain: https://filter-website.company/dns-query for solution verification.

Current laboratory solution and environment is well documented and correct as of writing time, changes may occur as both OpenSSL and nginx with ECH capabilities were obtained from non-official GitHub repositories. The specified laboratory solution is not the only implementation option available and does not include the use of proxy servers. ECH maximises user privacy with addition of proxies or relays which is currently lacking in the laboratory setup. Decisions for not implementing proxies or relays are based on such implementation would only be time costly and not affecting the final outcome in any way. In other words, the current laboratory environment is not utilising any forms of proxies as direct connections are being made from server's IP address to requestor's IP address. Real world application of ECH would rely on CDNs or proxy servers, in turn this would not make the public IP address of the web server visible in the destination IP address field for eavesdroppers, such threat actors would be required to have further network access such as between proxy server and target web server to obtain this information. Regardless of this factor, the solution works not by inspecting the destination IP address but rather by specified rulesets in DNS RPZ:s. The laboratory web server setup is in no way a complete rendition of ECH implementation, but rather a test environment for how ECH and revolving factors would react to the content filtering solution. Authors deem that selected implementation which is lacking partial functionality, may introduce validity threats, thus websites out of author's control will be included for the verification process. Authors cannot verify how each

external web server is set up, thus a variation of credible ECH-enabled websites will be tested. Validity and reliability concerns can be raised against the testing process not including all types of IT-infrastructure, thus more thorough testing is required once ECH has been adopted en masse. To reiterate, ECH status will be verified but if the external web servers are utilising proxies, rotating keys or incorporating additional features is difficult to verify.

The primary purpose for creating a web server with ECH capabilities under authors' control is to conduct the implementation in an accurate, ethical and scientific way. By eliminating potential third parties and expanding data points, the data presented is more accurate as external factors are excluded. One primary issue if the entire laboratory setup is to be conducted internally, meaning no use of VPS, is the issues with self-signed-certificates as modern browsers do not accept them as valid certificates. For the certificate issuer used, public IP addresses are required for verification of domain ownership. External ECH-enabled websites were used to further verify the solution, these websites do not host any content authors' deem as censorship worthy and was mainly used for testing purposes.

To utilise the function of ECH, it is required to rotate ECH keys, both private and public keys. The prime reason for rotating the ECH keys is to prevent active surveillance by adversaries. Due to the time constraint, the decision was made not to build rotation of keys in the laboratory environment.

5.2 Implementation of Solution

The subchapter explains in detail how the solution was implemented, specifically how the DNS filter was implemented and configured to work as a production server. Table 5 shows what applications have been installed, the IP address for the DNS filter as well as the chosen operating system for the DNS filter.

IPv4	Software	OS
70.34.201.112	BIND9	Debian 12
	Certbot	
	nginx	

Table 5. RPZ DNS server configuration

For specifically blocking ECH-enabled websites, the previously mentioned RPZ was implemented in the solution. Table 5 shows the server's application requirements. Firstly, it was needed to configure named.conf.options. The DNS filter is required to have DoH capabilities as mandated by ECH, DoH configuration has been previously explained for master DNS, revisit Appendix J and Appendix H for Let's Encrypt ACME challenges. See Appendix S for the full configuration of DNS filter DoH and RPZ in the named.conf.options file.

Next was to change the named.conf.local file to add the zone rpz.local which is used for defining the database file that contains records of websites that should be blocked, redirected, permitted and so on. See Appendix M for the RPZ zone configuration file.

Next was to create a database file containing RPZ records, RPZ specifies how the DNS filter should respond to a query. To create records, copy the db.empty template file, and name the file db.rpz.local, copy the template to the RPZ database file by the following

command: cp /etc/bind/db.empty /etc/bind/db.rpz.local. The db.rpz.local defines RPZ records e.g. domain defo.ie being blocked with a dot attribute, where a dot corresponds to domain being blocked. For more technical information regarding how DNS records were set up for the RPZ, see Appendix N.

As part of the RPZ, logging capabilities were implemented in the infrastructure. RPZ logs were used for logging trigger conditions, as well as troubleshooting purposes. For more technical information on how logging was implemented, see Appendix O.

Note that for db.rpz.local file changes, the serial needs to be increased by plus one for each change being saved in order to load a new serial of changes. To check the database configuration, the command is: named-checkzone rpz /etc/bind/db.rpz.local.

To verify that the solution works, the dig command can be used by typing dig @127.0.0.1 defo.ie, which returns NXDOMAIN in this case and surfing to the website secret.ech-website.company results in redirection.

6 Results

During verification of solution results, different web browsers and versions were used. Web browsers were chosen by popularity and ECH support, with one control browser not supporting ECH. This is to verify the solutions ability to coincide with the current landscape of non-ECH-enabled IT-infrastructure. For ECH support, notable issues were observed with Microsoft Edge not working as intended when interacting with the custom DNS filtering solution on Windows 11 operating systems. However, issues can arise when the DNS filtering solution is applied to Microsoft Edge in browser settings, an error is displayed in relation to the inability of verifying the DNS as an external vendor. The issue can be circumvented by applying the DNS filtering solution as a DoH server on the OS level.

Table 6 is a control test that was issued to verify solution functionality so it would not block access to all websites. The subdomain crypto.cloudlfare.com was added to RPZ database, specifying to block total access to the subdomain with the dot attribute. Proceeding this action, cloudflare.com was accessed with no issues observed. Notably, cloudfare.com does not support ECH so this test was done yet again but with the website created in subchapter 5.1 implementation of laboratory environment. Regarding ECH status on the blocked website crypto.cloudflare.com, no integrity of ECH was broken, ECH functionality was not established due to DNS response policy to not resolve the client requested query, resulting in NX domain response. Firefox ESR shows ECH status as inactive for website secret.ech-website.company due to lacking ECH support, the website was still accessible.

Control Websites With RPZ Active	In DNS RPZ	Website Accessibility	ECH Status Active	ECH Status Active Firefox ESR
cloudflare.com/	False	True	False	False
secret.ech-website.company	False	True	True	False
crypto.cloudflare.com/cdn-cgi/trace	True	False	False	False

Table 6. Control test of RPZ

Table 7 shows a list of current ECH-enabled websites as well as accessibility results, each listed website was defined in the DNS filter RPZ to be blocked from resolving with the dot attribute. Note that there are plenty of options for response policy as a redirect is also plausible. In other words, the list of ECH-enabled websites mentioned in Table 7, were blocked and not accessible. Verification for the accessibility of each website was done by running all six browsers mentioned in chapter 5 Table 1 and visiting the specified website. If all listed browsers did not find listed websites by the DNS filtering solution https://filter-website.company/dns-query, the column “Website Accessibility” would state “False”. The only caveat relates to the aforementioned browser Microsoft Edge, the DNS filter https://filter-website.company/dns-query was not accepted as a valid DoH server by the

browser, this was circumvented by applying the aforementioned DNS filter as a local DoH server on the Windows 11 client. When listed websites were not accessible, ECH would not be established, where column “ECH Status Active” describes the state with “False”.

ECH enabled Website	In DNS RPZ	Website Accessibility	ECH Status Active
secret.ech-website.company	True	False	False
Defo.ie	True	False	False
tls-ech.dev	True	False	False
research.cloudflare.com	True	False	False
crypto.cloudflare.com/cdn-cgi/trace	True	False	False

Table 7. Verification of defined RPZ zones.

Table 8 shows a list of ECH-enabled websites when listed websites were not defined in DNS RPZ. All six browsers were tested to verify website accessibility and ECH status. Note that Firefox ESR will not establish ECH due to lacking support, Firefox ESR was still able to access ECH-enabled websites which are not included in DNS filter RPZ regardless, this is due to backwards compatibility functionalities in ECH.

To verify ECH status for each website, Wireshark was used to inspect incoming and outgoing packets. Wireshark is an open source network monitoring tool used to inspect packets sent in a network, for this application, the Windows 11 local network interface card was monitored for establishment of ECH. To obtain and verify ECH status, search for ClientHello packet in monitored Wireshark session and inspect the extension data field, see examples in Appendix V. Due to some browsers not displaying the ECH flag even though the connection is valid, ECH verification on such browsers was done with Wireshark. Chromium based browsers, such as Google Chrome, Brave and Opera GX have built in features for this application, displaying ECH status through the “Encrypted ClientHello” flag in the “Security” tab. For more information regarding obtaining the “Encrypted ClientHello” flag, refer to the Appendix T for a step-by-step guide.

ECH enabled Website	In DNS RPZ	Website Accessibility	ECH Status Active	ECH Status Active Firefox ESR
secret.ech-website.company	False	True	True	False
Defo.ie	False	True	True	False
tls-ech.dev	False	True	True	False
research.cloudflare.com	False	True	True	False
crypto.cloudflare.com/cdn-cgi/trace	False	True	True	False

Table 8. Verification of website accessibility via the DNS filter

Now that all data has been compiled, Fisher's exact test can be computed. In Table 9, we can observe the consolidated data where each website is included in DNS RPZ and excluded for one iteration each.

ECH enabled Website	In DNS RPZ	Website Accessibility
secret.ech-website.company	False	True
Defo.ie	False	True
tls-ech.dev	False	True
research.cloudflare.com	False	True
crypto.cloudflare.com/cdn-cgi/trace	False	True
secret.ech-website.company	True	False
Defo.ie	True	False
tls-ech.dev	True	False
research.cloudflare.com	True	False
crypto.cloudflare.com/cdn-cgi/trace	True	False

Table 9. Consolidated RPZ/Website Accessibility Data

There are two variables at play which are of interest to disprove the null-hypothesis. RPZ is the independent variable and will be described as X, website accessibility is the dependent variable and will be described by the Y variable.

To calculate if there is a non-random statistically significant relationship between the two variables, the calculation of a p-value is necessary. The True variable will be translated into binary as 1 while the False variable will be translated to the binary value of 0.

X = In DNS RPZ	Y = Website Accessibility
0	1
0	1
0	1
0	1
0	1
1	0
1	0
1	0
1	0
1	0

Table 10. Binary results

A contingency table can now be created, documenting observations for each variable such as occurrences of values and including total number of rows and columns.

	Y = 0 (not accessible)	Y = 1 (accessible)	Row Total
X = 0 (RPZ Excl.)	0	5	5
X = 1 (RPZ Incl.)	5	0	5
Column Total	5	5	10

Table 11. Data contingency table

The following observations can be made for the variables required for calculating the hypergeometric probability for Fisher's exact test of the observed table.

a= 0 (number of occurrences where x = 0 and y = 0)
b = 5 (number of occurrences where x = 0 and y = 1)
c = 5 (number of occurrences where x = 1 and y = 0)
d = 0 (number of occurrences where x = 1 and y = 1)
n = 10 (number of observations)

Fisher's exact test calculates a p-value in a 2x2-5x5 contingency table, Fisher's exact test employs hypergeometric probability calculations where the p-value is used to examine if there is a non-random statistically significant relationship between an independent and dependent variable. The following formula was employed for the 2x2 contingency table (PennState Eberly College of Science, n.d.-a).

$$p = \frac{\binom{a+b}{a}\binom{c+d}{c}}{\binom{n}{a+c}}$$

Values from the dataset are inserted into the hypergeometric probability formula.

$$p = \frac{\binom{0+5}{0}\binom{5+0}{5}}{\binom{10}{0+5}}$$

Binomial coefficients are required to be calculated, the binomial coefficient formula will be employed to calculate each binomial coefficient.

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$\binom{5}{0} = \frac{5!}{0!(5-0)!} = \frac{5!}{0!*5!} = \frac{120}{1*120} = 1$$

$$\binom{5}{5} = \frac{5!}{5!(5-5)!} = \frac{5!}{5!*0!} = \frac{120}{120*1} = 1$$

$$\binom{10}{5} = \frac{10!}{5!(10-5)!} = \frac{10!}{5!*5!} = \frac{3628800}{14400} = 252$$

Now that all binomial coefficients have been calculated, final calculation can be completed to obtain the p-value.

$$p = \frac{1*1}{252} = \frac{1}{252} \approx 0.00397$$

7 Discussion

This research paper aimed to reduce knowledge gaps present regarding ECH whilst providing documentation on how to manage ECH in an organisational environment. A core building stone of how ECH management is to be addressed by organisations, was completed by proposing a solution on how to moderate client access to ECH-enabled websites. The thesis proposed two hypotheses where H0 is a null hypothesis and H1 is the alternative hypothesis, they are as follows:

H0: It is not possible to implement DNS filtering for moderating access to ECH-enabled websites

H1: It is possible to implement DNS filtering for moderating access to ECH-enabled websites

Findings observed in chapter 6 Results indicate a rejection of the null hypothesis H0 and show support of the alternative hypothesis H1. Changes implemented in the DNS filter RPZ successfully fulfil the requirements for moderating client access to ECH-enabled web content. Using hypergeometric probability calculations, a p-value of approximately 0.00397 was obtained. If the p-value is less than the common significance level of 0.05, the results strongly suggest that there is a statistically significant non-random relationship between the independent variable DNS filter RPZ, and the dependent variable website accessibility. Hypergeometric probability calculation was employed in Fisher's exact test to observe the probability that the presented data of small sample size occurred by chance. Given a p-value of 0.00397 and assuming that the null hypothesis is true, this indicates that there is only a 0.397% probability of observing such extreme data if the null hypothesis stands true. Given an α level of 0.05 and a p-value of approximately 0.00397, the results suggest H0 to be rejected, showing strong support for H1.

This study's main objective is to test the possibility of implementing DNS filtering for moderating access to ECH-enabled websites; no confidence interval nor odds ratio was calculated as that is not important for this research paper. To answer if H0 is true or false, Fisher's exact test is enough as the primary required data to answer the null hypothesis is to observe if there is an association present between selected variables, thus making Fisher's exact test adequate.

The solution allowed access to ECH-enabled websites which were not defined in the DNS RPZ configuration file, meanwhile blocking access to domains found in RPZ. Findings show that all domains defined in RPZ result in the client not being able to access the ECH-enabled websites. There was no compromise of ECH functionality due to the filtering solution utilising DoH, which is a requirement for clients to establish ECH-enabled TLS 1.3 connections.

In chapter 6 Results, it is possible to observe that the DNS filter is able to coincide with non-ECH-enabled infrastructure which should come as no surprise. All attempts to block website access succeeded where a non-existent domain response was issued if a blocked website was attempted to be accessed. Control tests were issued to verify overarching functionality so DNS filtering efforts would not interfere with regular operations. With the flexibility of the solution, it is possible to redirect users to alternative web servers. This can be used to track which client attempted to visit a prohibited website or to fine-tune access controls. If a request for a website is made frequently, system administrators have possibilities to observe such information and re-evaluate current filtering policies put in place. Note that when attempting to redirect clients from HTTPS enabled websites, connection will display to be insecure as the certificate does not match. Redirection was only replicated to a HTTP web server.

Reducing ECH knowledge gaps present was done by introducing core ECH functionality in the background section of the report. By presenting building stones for ECH and its main purpose, a broader insight in problems which can occur for organisational networks has been presented. However, there are still knowledge gaps present, these knowledge gaps are mainly attributed to ECH being a niche subject, and the authors have made an effort to reduce these knowledge gaps. This report is not definitive, as only a reduction has been made by introducing potential problems which can arise with careless adoption of the TLS 1.3 extension ECH.

DNS filtering is indicated to be the most promising option for content moderation in organisational environments with ECH in mind. Middleboxes were discussed throughout this report to be expensive solutions which degrade network performance substantially (Hamilton et al., 2020). While the supporting infrastructure required for DNS filtering to be feasible can vary, the outright advantage in being able to make decisions without performing complex operations such as MitM attacks, nor analysing every incoming and or outgoing HTTPS packet, is a substantial advantage for DNS filtering.

The presented solution is highly scalable as this research suggests it works not only for ECH-enabled websites but also ESNI, HTTPS and more. The solution is mostly applicable in organisational environments due to the native high control of network and system IT-infrastructure. DNS filtering is nothing which is implemented fully in house for private use as if such requirements are present, external DNS filters are employed. DNS filtering in itself is nothing new but as presented by this research, is the most prevalent option for ECH content filtering. This is as previously mentioned due to CDNs and or proxy servers not presenting information on the internet layer which can be directly tied to the target web server. If collateral damage is to be accounted for, and the SNI data field is not accessible for inspection, DNS filtering remains the most valid option available for mass content moderation.

The solution is primarily suitable for organisational environments or as an implementation option for system administrators to govern what content clients are able to access. Overarching network and IT-infrastructure control is required for DNS filtering to be effective, as many prevalent circumvention options are available if system and network access is obtained with basic privileges (Surveillance Self-Defense, n.d.). Such a solution would require to oversee VPN access, proxies which can circumvent DNS filters and abilities of changing specified DNS.

VPNs can circumvent the DNS filter all together, using differentiating DNS servers which are dictated by the VPN provider once connected. This causes headaches for network- and system administrators since VPNs can be a valuable tool used for many applications. Proxy servers can act like intermediaries, executing DNS queries on behalf of the client. This works similarly like VPNs and bases itself on the same circumvention technique, bypassing the DNS filter and using a differentiating DNS server.

The Microsoft Edge web browser did detect the custom DNS filter proceeding updates, however as a result, it can arise a challenge where Microsoft Edge does not detect the specified custom DoH replicated on Windows 11. Microsoft Edge running on Windows 10 was able detect the DNS filter once specified in the web browser settings. When Microsoft Edge did not detect the DNS filter, the issue was circumvented by setting local machine DNS to the DNS filter and using local DNS settings on Microsoft Edge. Root of cause for this issue is unknown where as on Microsoft Edge version 124.0.2478.80, the DNS filter was recognized as a valid external DoH provider.

To quantify the effectiveness of DNS filtering in relation to ECH, additional research is required comparing the presented solution to existing solutions such as middleboxes, which rely on specialised hardware and software. It is important to keep in mind that DNS filtering requires additional supporting implementations against circumvention to be deemed effective.

There are some validity threats which should be addressed. The use of stale cryptographic keys in forward lookup DNS records is an issue and in no way reflects a complete implementation of an ECH enabled website. Stale keys cause a numerous amount of security and validity implications but for the test environment they cause no hindrance for the end result. As the DNS filter has no knowledge if the cryptographic keys are stale or not, no functional difference is observed. The same argument can be added to the lack of a proxy server in front of the target web server. In real world applications, to get full use of ECH, the target website has to be hosted behind a proxy server so network eavesdroppers are not able to identify the target server according to the destination IP address. In the current configuration of the laboratory environment, it is possible to implement IP filtering against the website without any collateral damage as there is only one IPv4 address linked to the web server. In real world applications, adding the complexity which CDNs introduce, this option would not be as effective and cause other web services to be restricted if IP filtering is to be employed.

DNS filtering is suggested to be the simplest solution for moderating HTTPS type traffic. The SNI field being encrypted only adds to the value of DNS filtering solutions. By creating a recursive DNS server between a client and a DNS server which may have the record on file, logging and creating access controls can be ideally placed. Since SNI information is obtained from DNS servers, placing a recursive DNS server with filtering abilities theoretically intercepts any client-made queries before they are sent further along the DNS chain. This is an advantage to previously mentioned middleboxes which are specialised appliances which are configurable to intercept HTTPS packets. Such solutions are not only costly monetarily but also costly in relation to network throughput (Hamilton et al., 2020). If a device or a cluster of devices need to intercept and inspect each HTTPS packet, this will lead to hindrance of network flow which in turn degrades network performance. DNS filters on the other hand, do not require to inspect each HTTPS packet since the query is directly sent to the filter. Depending on scope, authors suggests that hypothesis

H1: It is possible to implement DNS filtering for moderating access to ECH-enabled websites stands true.

7.1 In Relation to Previous Research

Trevisan et al. (2020, 2023) suggest that DoH is insecure due to machine learning models being able to predictively determine which websites clients are visiting with high success rates. This research accurately represents current IT-infrastructure with a mixture of DoH and unencrypted DNS traffic observable on the network. It is suggested that fully forcing all clients to use DoH would hinder the effectiveness of machine learning models used for predicting which websites clients are visiting, such efforts would attempt to hinder the process of obtaining an accurate data set to train the machine learning models on (Trevisan et al., 2023). If DoH is forced on system and network level, no unencrypted DNS traffic will be observable for training purposes in relation to machine learning models. The solution presented is based on DoH with filtering abilities, all clients would make their queries through the DNS filter over DoH in compliance to mitigating observable unencrypted DNS traffic.

In relation to side-channels leaking information regarding which website the client is connecting to, the test environment does not have any side-channels, thus eliminating such risks (Khandkar et al., 2023). Due to the simplicity of the implemented website, side-channels are of no use. If the requirements of side-channels are present, use only TLS 1.3 ECH-enabled side-channels so as to not undermine user privacy.

Regarding observable collateral damage for websites on Alexa top 1M, concerns should be raised against this datapoint as it does not accurately represent the true scale of collateral damage. Alexa top 1M represents the one million most popular websites according to Amazon, one could suggest that these websites have substantial funding and ability of recognizing if they have been affected by collateral damage, taking matters in their own hands and changing IP address or proxying server IP address as to circumvent accidental IP blockages. Problems still stand where extensive knowledge is required if an attempt of performing IP blocking efforts towards a proxy server or CDNs public IP addresses is to be executed.

DPI firewalls or clusters of DPI firewalls cause reduced network speed and or increased latency (Hamilton et al., 2020). This is explained due to the excessive overhead required to perform DPI operations, causing potential congestion and slowing down the network. A network is only as fast as its slowest link, meaning that if DPI firewalls or other types of middleboxes experience heavy load, and the only link between the outside network and inside network goes through a middlebox, the network will experience degraded performance. Depending on configuration, all packets are usually examined by a DPI firewall. As previously mentioned, DNS filtering selectively obtains DoH packets which are usually indistinguishable compared to HTTPS network traffic. DNS filtering retains a massive advantage in simplicity, where it does not require to inspect each HTTPS packet as the DoH query is being directly sent to the DNS filter.

7.2 Ethical and Societal Aspects

ECH is rapidly being developed and is in early stages of RFC publication with few ethical concerns ahead. Firstly, there is a risk of not being held accountable for visiting an ECH website which may not be permissible. This was addressed in the implementation section with DNS filtering for ECH-enabled websites.

Web content filtering methodologies are used for restricting access for websites which may not be permissible through policies or guidelines, these methodologies could also be used interchangeably for mass censorship (Hamade, 2008), this could be problematic and raise ethical concerns as the internet thrives on freedom of information and freedom of speech. Websites host a vast majority of content across the internet, catering to all types of individuals. There are select websites which should not be accessible based on morality and or legality (Hamade, 2008), but censorship methodologies are often abused for purposes which may be outside of the general populous interest (Satija & Chatterjee, 2021). How censorship methodologies are used is quite difficult to put restraints on, meaning publications which can be related to censorship should be treated with care (Hamade, 2008). A consensus should be kept in mind that such publications can be used for negative attributions for society.

When it comes to ethical and societal aspects in subjects relating to web based content filtering solutions, there are usually concerns to be adjusted for. This research suggests that DNS filtering is the primary method for dealing with web content moderation if collateral damage from IP filtering is to be accounted for (Chai et al., 2019). DNS filtering is only effective if high system and network restrictions are put in place as circumvention techniques are prevalent and effective (Feeney et al., 2004; Hamade, 2008). Authors suggest that no indications can be found that DNS filtering in relation to ECH web content, would be applicable for censorship en masse, but rather in highly controlled IT-environments. This is due to the level of control which is required, and is rather difficult to obtain, as it heavily infringes on the general populations' privacy. Research does not rule out the possibility of such censorship efforts being attempted, rather suggests the difficulty of implementing such restrictions effectively.

The various circumvention methodologies which are present, and the relatively low effort required to succeed with implementing circumvention techniques (Surveillance Self-Defense, n.d.), deems DNS filtering to not be a valid mass censorship methodology. DoH does not allow for DNS hijacking which is a prevalent censorship methodology used by the *Great Firewall* (GFW) of China, excluding this censorship vector (Chai et al., 2019). An attempt to implement DNS filtering en large if DoH is popularised, would be easily circumvented by citizens through simply changing DNS provider manually in the web browser (Satija & Chatterjee, 2021). Usage of VPNs or proxy servers to contact a non restricted DNS server would be a valid circumvention technique. Note that through extreme lengths it could be possible to force a large chunk of users to employ regulated DNS servers, as not all internet users are educated in their privacy nor care.

DNS filtering is nothing new and has been used for a long time (Cheswick & Bellovin, 1996), IP filtering has been the most prevalent censorship methodology applied due to the general effectiveness in removing access to content on the internet. This is mostly due to not only the effectiveness but also ease of implementation as the requirements are quite low for such efforts.

The research paper covers two UN SDGs, goal 4 ensures inclusive and equitable quality education and promotes lifelong learning opportunities for all (United Nations, n.d.). The target indicator 4.4.1 and target 4.c covers specifically the aim of this research paper. The target 4.4.1 is defined as the proportion of youth and adults with information and communications technology by the type of skill.

Another aspect covered by the research paper and supported by UN SDGs is goal 9, striving for building resilient infrastructure promoting sustainable and inclusive industrialization and fostering innovation. UN SDG target 9.5 covers specifically enhancing scientific research, in this case enhancing privacy, security considerations as well as addressing current challenges with wide ECH adoption. The goal 9.5 further discusses upgrading technological capabilities of industrial sectors in all countries. Furthermore, by 2030, the number of research and development workers will be substantially increased per one million people as well as encouraging innovation, which part of this research paper is for encouraging wide ECH adoption and expediting the quality assured RFC publication for the ECH (United Nations, n.d.).

8 Conclusion

To conclude, this study aimed for reducing knowledge gaps for ECH and providing documentation on how to manage ECH in an organisational environment, and proposing a solution on moderating access to ECH-enabled websites. Basic ECH website configuration was presented where core functionality was examined with success. The created website with the addition of ECH-enabled websites out of authors' control, were tested against a DNS filtering solution. The most popular web browsers were used when conducting the verification process to get a broader understanding of how browsers might interact with the solution. All five ECH enabled web browsers and one non-ECH supported web browser acted according to expectations when interacting with the solution, recognizing the DNS filter, which resulted in the client not being able to access websites restricted by filtering efforts. Data presented in chapter 6 Results, shows a direct relation where if a response policy record can be found in the DNS filter relating to requested website, the website would not be accessible, giving a non-existent domain response by DNS filter to the client. Backwards compatibility was tested where the solution functioned without any issues during the verification process with non-ECH-enabled web browsers, this is very important as the current IT-landscape is diverse, where browsers can be outdated or simply do not support ECH. The study addresses partial security concerns which are presented by the research community, where ECH flaws have been examined and remedies have been accredited accordingly. The study suggests that hypothesis *H1: It is possible to implement DNS filtering for moderating access to ECH-enabled websites* stands true.

The DNS filtering solution presented in the report does not depict a fully complete implementation as there are additional steps required to deal with circumvention techniques. DNS filtering efforts are vulnerable if network and system administrative controls are readily available for end-users. Controls and revisions for accessing proxy servers, VPNs and browser settings need to be restrained for clients as DNS resolutions can be obtained from nameservers out of network- and system administrators controls. Due to the difficulty of measuring which kind of DNS filtering circumvention restrictions are required, depending on IT-infrastructure topology, it is difficult to quantify existing to presented solution effectiveness. Authors suggest that DNS filtering is the most prevalent solution for ECH-enabled web content moderation in an organisational environment, if collateral damage is to be accounted for.

8.1 Future Work

Research is based on ECH datatracker, lacking official RFC publication. A revision would be required once official ECH RFC has been published as to verify if any changes made to ECH has affected solution functionality and effectiveness. The responsibility of CDNs in relation to ECH adoption should be studied as a shift in obligations can be subject for discussions. A comparison of solution effectiveness and implementation costs can be a study, where specialised appliances such as middleboxes can be put to the test against DNS filtering in differentiating environments, for example in corporations, home environments and mass scale filtering applications with ECH in mind. As the effectiveness of the solution has to incorporate the cost of implementing circumvention restrictions, it can be difficult to quantify

effectiveness due to networks and system topology differentials. If ECH adoption is to be expected, a study could be attempted to answer concerns relating to ECH circumventing content filtering precautions. Studies have been made theorising which effects ECH could imply on current content filtering efforts (Satija & Chatterjee, 2021), but is lacking real world application data. These studies have been conducted on national geographical areas, but it would be of high interest to observe real world ECH interaction with diverse environments and a more precise dataset of websites.

References

Alani, M. M. (2014). *Guide to OSI and TCP/IP Models*. Springer International Publishing. https://doi.org/10.1007/978-3-319-05152-9

Chai, Z., Ghafari, A., & Houmansadr, A. (2019). On the Importance of Encrypted-SNI (ESNI) to Censorship Circumvention. *USENIX Association*. https://www.usenix.org/conference/foci19/presentation/chai

Cheswick, B., & Bellovin, S. M. (1996). *A DNS filter and switch for packet-filtering gateways*. 6th USENIX Security Symposium 1996. Scopus. https://www.usenix.org/legacy/publications/library/proceedings/sec96/full_papers/cheswick/index.html

Cloudflare. (2023, October 11). *Early Hints and Encrypted Client Hello (ECH) are currently disabled globally*. https://community.cloudflare.com/t/early-hints-and-encrypted-client-hello-ech-are-currently-disabled-globally/567730

Debian Wiki. (n.d.). *Bind9*. Debian Wiki. https://wiki.debian.org/Bind9#File_.2Fetc.2Fbind.2Fnamed.conf.options

Dierks, T., & Rescorla, E. (2008). *The Transport Layer Security (TLS) Protocol Version 1.2* (RFC5246; p. RFC5246). RFC Editor. https://doi.org/10.17487/rfc5246

Droms, R. (1997). *Dynamic Host Configuration Protocol* (RFC2131; p. RFC2131). RFC Editor. https://doi.org/10.17487/rfc2131

Eastlake, D. (2011). *Transport Layer Security (TLS) Extensions: Extension Definitions* (RFC6066; p. RFC6066). RFC Editor. https://doi.org/10.17487/rfc6066

Feeney, K. C., Lewis, D., & Wade, V. P. (2004). Policy based management for Internet communities. *Proceedings. Fifth IEEE International Workshop on Policies for Distributed Systems and Networks, 2004. POLICY 2004.*, 23–32. https://doi.org/10.1109/POLICY.2004.1309147

Friedl, S., Popov, A., Langley, A., & Stephan, E. (2014). *Transport Layer Security (TLS) Application-Layer Protocol Negotiation Extension* (RFC7301; p. RFC7301). RFC Editor. https://doi.org/10.17487/rfc7301

Goldlust, S. (2021). *DNS Over HTTPS With BIND 9.17*. https://www.isc.org/blogs/doh-talkdns/

Göppert, J., Chomel, A., E, J. S., & Sikora, A. (2023). Performance Evaluation of TLS Session Establishments on an ARM Cortex-M4 Platform. *2023 IEEE 12th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*, 1205–1210. https://doi.org/10.1109/IDAACS58523.2023.10348947

Guo, R., Chen, J., Liu, B., Zhang, J., Zhang, C., Duan, H., Wan, T., Jiang, J., Hao, S., & Jia, Y. (2018). Abusing CDNs for Fun and Profit: Security Issues in CDNs' Origin Validation. *2018 IEEE 37th Symposium on Reliable Distributed Systems (SRDS)*, 1–10. https://doi.org/10.1109/SRDS.2018.00011

Hamade, S. N. (2008). Internet Filtering and Censorship. *Fifth International Conference*

on Information Technology: New Generations (Itng 2008), 1081–1086. https://doi.org/10.1109/ITNG.2008.50

Hamilton, R., Gray, W., Sibanda, C., Kandasamy, S., Kirner, R., & Tsokanos, A. (2020). Deep Packet Inspection in Firewall Clusters. *2020 28th Telecommunications Forum (TELFOR)*, 1–4. https://doi.org/10.1109/TELFOR51502.2020.9306651

Hoffman, P., & McManus, P. (2018). *DNS Queries over HTTPS (DoH)* (RFC8484; p. RFC8484). RFC Editor. https://doi.org/10.17487/RFC8484

ISC. (n.d.). *Tutorial on Configuring BIND to use Response Policy Zones (RPZ)*. ISC. https://www.isc.org/docs/BIND_RPZ.pdf

Khandkar, V. S., Hanawal, M. K., & Kulkarni, S. G. (2023). *State of Internet Privacy and Tales of ECH-TLS*. 165–170. Scopus. https://doi.org/10.1109/COMSNETS56262.2023.10041275

Koshy, A. M., Yelluru, G., Moharir, M., Deepamala, N., & Ramakanth Kumar, P. (2023). Privacy Leaks Via SNI and Certificate Parsing. *2023 International Conference on Quantum Technologies, Communications, Computing, Hardware and Embedded Systems Security (iQ-CCHESS)*, 1–5. https://doi.org/10.1109/iQ-CCHESS56596.2023.10391827

Kraus, L., Ukrop, M., Matyas, V., & Fiebig, T. (2020). Evolution of SSL/TLS Indicators and Warnings in Web Browsers. In J. Anderson, F. Stajano, B. Christianson, & V. Matyáš (Eds.), *Security Protocols XXVII* (Vol. 12287, pp. 267–280). Springer International Publishing. https://doi.org/10.1007/978-3-030-57043-9_25

Li, X., Azad, B. A., Rahmati, A., & Nikiforakis, N. (2021). Good Bot, Bad Bot: Characterizing Automated Browsing Activity. *2021 IEEE Symposium on Security and Privacy (SP)*, 1589–1605. https://doi.org/10.1109/SP40001.2021.00079

Merget, R., Brinkmann, M., Aviram, N., Somorovsky, J., & Mittmann, J. (2021). Raccoon Attack: Finding and Exploiting Most-Significant-Bit-Oracles in. *USENIX Association*, 213--230. https://www.usenix.org/conference/usenixsecurity21/presentation/merget

Nginx. (n.d.). *Building nginx from Sources*. Nginx. https://nginx.org/en/docs/configure.html

PennState Eberly College of Science. (n.d.-a). *Fisher's Exact Test*. PennState Eberly College of Science. https://online.stat.psu.edu/stat504/lesson/4/4.5

PennState Eberly College of Science. (n.d.-b). *Significance Levels*. PennState Eberly College of Science. https://online.stat.psu.edu/stat200/lesson/6/6.2

Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (RFC8446; p. RFC8446). RFC Editor. https://doi.org/10.17487/RFC8446

Rescorla, E., Kazuho, O., Sullivan, N., & Wood, C. (2024). *TLS Encrypted Client Hello*. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-tls-esni/18/

Satija, S., & Chatterjee, R. (2021). BlindTLS: Circumventing TLS-based HTTPS censorship. *Proceedings of the ACM SIGCOMM 2021 Workshop on Free and Open Communications on the Internet*, 43–49.

https://doi.org/10.1145/3473604.3474564

Savin, F. (2023, November 27). *Setting Up ECH for a Website* [CUJO AI]. https://cujo.com/blog/set-up-ech-website/

Schäferhoff, N. (n.d.). Web Hosting Statistics Market Share, Trends and Biggest Companies. *WebsiteSetup*. https://websitesetup.org/news/web-hosting-statistics/

Shamsimukhametov, D., Kurapov, A., Liubogoshchev, M., & Khorov, E. (2022). Is Encrypted ClientHello a Challenge for Traffic Classification? *IEEE Access*, *10*, 77883–77897. https://doi.org/10.1109/ACCESS.2022.3191431

Shamsimukhametov, D. R., Kurapov, A. A., Liubogoshchev, M. V., & Khorov, E. M. (2023). Indistinguishability of Traffic by Open TLS Parameters with Encrypted ClientHello. *Journal of Communications Technology and Electronics*, *68*(12), 1523–1529. https://doi.org/10.1134/S1064226923120173

Sharma, A. (2021, July 16). *Critical Cloudflare CDN flaw allowed compromise of 12% of all sites*. https://www.bleepingcomputer.com/news/security/critical-cloudflare-cdn-flaw-allowed-compromise-of-12-percent-of-all-sites/

Sling Academy. (n.d.). *NGINX stream core module: The Complete Guide*. Sling Academy. https://www.slingacademy.com/article/nginx-stream-core-module-complete-guide/

Surveillance Self-Defense. (n.d.). How to: Understand and Circumvent Network Censorship. *Surveillance Self-Defense*. https://ssd.eff.org/module/understanding-and-circumventing-network-censorship

Svenska Institutet för Standarder. (2022). *Information security, cybersecurity and privacy protection—Guidance on managing information security risks (ISO/ IEC 27005:2022, IDT)*. Svenska Institutet för Standarder. https://www.sis.se/produkter/foretagsorganisation/tjanster/ovrigatjanster/ss-isoiec-270052022/

Trevisan, M., Soro, F., Mellia, M., Drago, I., & Morla, R. (2020). Does domain name encryption increase users' privacy? *ACM SIGCOMM Computer Communication Review*, *50*(3), 16–22. https://doi.org/10.1145/3411740.3411743

Trevisan, M., Soro, F., Mellia, M., Drago, I., & Morla, R. (2023). Attacking DoH and ECH: Does Server Name Encryption Protect Users' Privacy? *ACM Transactions on Internet Technology*, *23*(1), 1–22. https://doi.org/10.1145/3570726

United Nations. (n.d.). *THE 17 GOALS*. United Nations. https://sdgs.un.org/goals

Van der Sar, E. (2023, October 6). Encrypted Client Hello (ECH) Effectively Defeats Pirate Site Blocking. *TorrentFreak*. https://torrentfreak.com/encrypted-client-hello-ech-effectively-defeats-pirate-site-blocking-231006/

Wong, K. H., Lau, P. Y., & Park, S. (2013). Using Surrogate Servers for Content Delivery Network Infrastructure with Guaranteed QoS. *Journal of Advances in Computer Networks*, 34–38. https://doi.org/10.7763/JACN.2013.V1.7

Yadav, T. K., Sinha, A., Gosain, D., Sharma, P. K., & Chakravarty, S. (2018). Where The Light Gets In: Analyzing Web Censorship Mechanisms in India. *Proceedings of the Internet Measurement Conference 2018*, 252–264. https://doi.org/10.1145/3278532.3278555

Yang, C.-S., Liao, M.-Y., Luo, M.-Y., Wang, S.-M., & Yeh, C. E. (2010). A Network Management System Based on DPI. *2010 13th International Conference on Network-Based Information Systems*, 385–388. https://doi.org/10.1109/NBiS.2010.71

Zirngibl, J., Sattler, P., & Carle, G. (2023). A First Look at SVCB and HTTPS DNS Resource Records in the Wild. *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 470–474. https://doi.org/10.1109/EuroSPW59978.2023.00058

Appendices

Appendices which are mentioned throughout the report are ordered in alphabetical order, some appendices contain additional explanation related to configuration files.

Appendix A - named.conf.options

This appendix represents configuration for master DNS located at: /etc/bind/named.conf.options.

Text within brackets is to be modified with the brackets removed.

```
options {
        directory "/var/cache/bind";
        forwarders {
                [dns_of_choice];
        };
        dnssec-validation auto;
        auth-nxdomain no; # conform to RFC1035
        listen-on-v6 { any; };
        allow-transfer {[slave_dns_ip]; };
        allow-query { any; };
        allow-recursion { any; };
};
```

Appendix B - named.conf.local

Appendix B shows the master DNS configuration for file located at: etc/bind/named.conf.options, as well as how the file is configured.

Text within brackets should be altered with the brackets removed.

```
zone "[34.70.in-addr.arpa]" {
        type master;
        file "/etc/bind/[db.IP_to_name]";
        allow-transfer {  [slave_ip_addr]; };
};


zone "[your_domain_here]" IN {
    type master;
    file "/etc/bind/[db.name_to_IP]"; // Path to your zone file
    allow-transfer { [slave_ip_addr]; };
    allow-update { none; }; // Disabling dynamic updates, adjust
as needed
};
```

Appendix C - Slave Config for named.conf.local

The appendix shows how the slave named.conf.local configuration file is configured.

```
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in
your
// organization
//include "/etc/bind/zones.rfc1918";

zone "34.70.in-addr.arpa" {
        type slave;
        masters{ 70.34.221.99; };
};


zone "ech-website.company" IN {
       type slave;
       masters { 70.34.221.99; };
};
```

Appendix D - Master db.name_to_IP

This appendix describes how DNS records are set up in the main DNS server. The below configuration shows how DNS records are made for master DNS.

```
$TTL    604800
@       IN      SOA     ns1.ech-website.company.
admin.ech-website.company. (
                              20240402         ; Serial
                              604800         ; Refresh
                              86400         ; Retry
                              2419200         ; Expire
                              604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.ech-website.company.
@       IN      A       70.34.223.34
ns1     IN      A       70.34.221.99
www     IN      A       70.34.223.34
```

Appendix E - Master Reverse db.name_to_IP

This appendix describes how DNS reverse lookup records are set up in the master DNS server.

```
$TTL    604800
@       IN      SOA     ns1.ech-website.company.
admin.ech-website.company. (
                              20240402          ; Serial
                              604800          ; Refresh
                              86400         ; Retry
                              2419200          ; Expire
                              604800 )        ; Negative Cache TTL
;
@       IN      NS      ns1.ech-website.company.
34      IN      PTR     ech-website.company.
```

Appendix F - OpenSSL Installation Guide

This appendix aims to guide the installation of OpenSSL from the aforementioned GitHub repository. To install OpenSSL navigate to openssl directory and type ./Configure. When configuration of OpenSSL installation has been done, it is important to execute the installation by running the following commands: make && sudo make install. Do not install OpenSSL as root due to potential errors which can occur. After the installation has been done, to check whether ECH implementation is working type in /usr/bin/local/openssl ech -h. If the installation process succeeded, it should prompt with the following results according to the below output:

```
Usage: ech [options]

General options options:
 -help           Display this summary
 -verbose        Provide additional output (though not much:-)

Key generation options:
 -pemout outfile   Private key and ECHConfig [echconfig.pem]
 -pubout outfile   Public key output file
 -privout outfile  Private key output file
 -public_name val  public_name value
 -mlen int       Maximum name length value
 -suite val      HPKE ciphersuite: e.g. "0x20,1,3"
 -ech_version int  ECHConfig version [0xff0d (13)]
 -extfile val    Input file with encoded ECH extensions


ECHConfig print/down-selection options:
 -pemin outfile File with optional private key and ECHConfig
 -select int    Output only n-th ECHConfig from input file
```

Next is to type the following commands to make the installation for OpenSSL the dev version and install it in the operating system: make && sudo make install. The installation might fail, due to some files being missed in OpenSSL installation in OS Linux Ubuntu 23.10.

Appendix F.1 - OpenSSL Installation Troubleshoot

This sub-appendix describes the troubleshooting process in case certain files were missing. The following files might be missing: libcrypto.a and/or libssl.a. To fix it, firstly should the directory .openssl be created, where inside a subdirectory lib be created. Proceed by moving the files to the following path with the mv command, the process should look like the following below:

```
mv /path/to/openssl/download/directory/filename.a
/pathopenssl/.openssl/lib/libcrypto.a && mv
/path/to/openssl/download/directory/filename.a
/pathopenssl/.openssl/lib/libssl.a
```

Appendix G - nginx Installation Guide

This appendix represents an installation guide on installing nginx from the aforementioned GitHub repository, mentioned in Chapter 4. Methodology. To install Nginx, it is required to do following AutoConfigure for the nginx as described in the configuration below:

```
./auto/configure           --with-debug           --prefix=/opt/ech
--with-http_ssl_module    --with-stream    --with-stream_ssl_module
--with-stream_ssl_preread_module
--with-openssl=/home/ech/src/openssl    --with-openssl-opt="--debug
'-Wl,--enable-new-dtags,-rpath,$(LIBRPATH)'" --with-http_v2_module
```

To parse these options: –-with-debug option is made for troubleshooting problems for configurations and log issues on the logging file. The option --prefix=/opt/ech defines where nginx will be installed. The next option --with-http_ssl_module provides the necessary support for the HTTPS protocol (Nginx, n.d.). The option --with-stream is a nginx stream core module for handling the TCP/UDP traffic, TLS terminations, load balancing while providing performance and reliability for the web server (Sling Academy, n.d.). Moving further with the option --with-stream_ssl_module which adds SSL/TLS support to the stream module. Similar option --with-stream_ssl_preread_module permits extracting information from ClientHello without interrupting SSL/TLS connection. OpenSSL needs to be integrated as well in order to utilise modules such as --with-stream_ssl_module which are dependent on the OpenSSL library. The option --with-openssl=/home/ech/src/openssl defines the path of the OpenSSL installation directory. The next parameter is --with-openssl-opt="--debug' -Wl,--enable-new-dtags,-rpath,$(LIBRPATH)'" defines the options to use a shared libraries path for OpenSSL with debug mode. Next add-on is the --with-http_v2_module, it provides HTTP version 2 module as well as ALPN.

When configuration has been done, next was to do the following commands in order to make the Nginx installation and install the application: make && sudo make install. Note that these commands need to be running under a user and not under root, however sudo is being used in order to execute the installation, otherwise installation might not complete due to lack of permissions as a user.

When ECH configuration has been verified, nginx configuration was required to be verified whether ECH support is built-in or not, by typing following commands in figure 13.

```
strings /opt/ech/sbin/nginx | grep -w ssl_echkeydir
```

If correctly configured the output should be as follows: ssl_echkeydir. The nginx should be started by the following command: sudo /opt/ech/sbin/nginx. For the troubleshooting process, when configuring nginx.conf, it is recommended to check the configuration by sudo /opt/ech/sbin/nginx -t and if configuration is correct, the following output will be shown in configuration below:

```
nginx: the configuration file /opt/ech/conf/nginx.conf syntax is ok
nginx: configuration file /opt/ech/conf/nginx.conf test is
successful
```

Appendix H - Completing ACME Challenge

In order to acquire SSL certificates, CertBot was used as a tool to retrieve certificates from Let's Encrypt. Note that Let's Encrypt has authentication requirements when requesting certificates and that is done by completing ACME challenge. ACME which involves DNS TXT records and web page verification when obtaining wildcard domain certificates. Requirements may differ from requested certificate types where only web page verification may be sufficient for the retrieval of certificates. For wildcard certificates the following line should be entered on the web server according to figure 15.

```
certbot certonly -d '*.ech-website.company' --expand --manual
```

A request for creating a file under file directory .well-known/acme-challenge/ will be made, this is to prove domain ownership where the files need to be created under /var/www/html directory according to the commands below:

```
ech@web-server: cd /opt/ech/html/
ech@web-server: sudo mkdir .well-known
```

It is important to change permissions of created files and directories under the html directory, for them to function properly, directories should have 755 permissions or rwx-rx-rx and files should have 644 or rw-r-r according to the following commands below:

```
ech@web-server: sudo chmod 755 .well-known
ech@web-server: cd .well-known
ech@web-server: sudo mkdir acme-challenge
ech@web-server: sudo chmod 755 acme-challenge
ech@web-server: sudo touch
tbRM9CLobsRl3Iw9xMG5A3-zWpk09gKbdFGnWRx538M
ech@web-server: sudo chmod 644
tbRM9CLobsRl3Iw9xMG5A3-zWpk09gKbdFGnWRx538M
ech@web-server: sudo nano
tbRM9CLobsRl3Iw9xMG5A3-zWpk09gKbdFGnWRx538M
```

Proceed with entering the string of data which is requested by CertBot and write out to the file. Verify if the file is accessible by visiting the URL requested by CertBot.

Appendix I - Permission Changes for Accessing SSL Certificates

This appendix explains in detail on what changes are required for BIND on DNS server to access certificate files in order for DoH to be functional.

```
ech@dns-server: sudo chmod 0755 /etc/letsencrypt/live
ech@dns-server: sudo chmod 0755 /etc/letsencrypt/archive

ech@dns-server:             sudo             chgrp             bind
/etc/letsencrypt/live/ns1.ech-website.company/privkey.pem
ech@dns-server:             sudo             chmod             0640
/etc/letsencrypt/live/ns1.ech-website.company/privkey.pem

ech@dns-server: sudo nano /etc/apparmor.d/local/usr.sbin.named
```

Permissions have been adjusted so that apparmor can read the certificates for DNS servers. The configuration below continues with sudo nano /etc/apparmor.d/local/usr.sbin.named where read permissions need to be delegated to the configuration file for DNS servers to work. Changes for the configuration file can be seen below.

```
/etc/letsencrypt/** r,
```

The output below shows when command sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.named, it reloads the configuration of the file usr.sbin.named.

```
ech@dns-server:          sudo          apparmor_parser          -r
/etc/apparmor.d/usr.sbin.named
```

Appendix J - DoH Configuration on DNS Server

This appendix represents how a DNS server should be configured according to the DoH configuration in

```
tls local-tls {
     key-file
"/etc/letsencrypt/live/ech-website.company/privkey.pem";
     cert-file
"/etc/letsencrypt/live/ech-website.company/fullchain.pem";
};

http local-http-server {
     # multiple paths can be specified
     endpoints { "/dns-query";  };
};

options {
        directory "/var/cache/bind";
        forwarders {
                1.1.1.1;
        };
        dnssec-validation auto;
        auth-nxdomain no; # conform to RFC1035
        listen-on-v6 { any; };
        allow-transfer { 70.34.201.228; };
        allow-query { any; };
        allow-recursion { any; };
        listen-on port 53 { any; };
        #Doh
        https-port 443;

        #IPv4 sektion
        # example for encrypted DoH
        # listening on all IPv4 addresses.
        listen-on port 443 tls local-tls http local-http-server
{any;}; ##specifying key and endpoint for DoH communication
};
```

Appendix K - index.html Configuration File

This appendix explains the index.html configuration file.

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<pre>
HTTP host: <!--# echo var="http_host" -->
ALPN protocol: <!--# echo var="ssl_alpn_protocol" -->
SSL cipher: <!--# echo var="ssl_cipher" -->
SSL protocol: <!--# echo var="ssl_protocol" -->
SNI: <!--# echo var="ssl_server_name" -->
ECH status: <!--# echo var="ssl_ech_status" -->
Outer SNI (public name): <!--# echo var="ssl_ech_outer_sni" -->
Inner SNI: <!--# echo var="ssl_ech_inner_sni" -->
</pre>
</body>
</html>
```

Each line of code within the pre tags echos a variable which is defined by a string within quotation marks. The first echo displays http_host which is the current hostname dictated by the domain name. Proceeding formention variable comes SSL data such as ALPN, specifying which version was chosen during the SSL handshake. SSL cipher and SSL protocol shows what cipher suite is used and what version of TLS is active. Regardless if observation is made with all requirements fulfilled for ECH or not, SSL protocol variable should display TLS version 1.3 as ECH is not mandated. Note that other variables may have issue retrieving data if ECH requirements are not fulfilled, these sections consist of ECH status, outer and inner SNI as they will display no data or an error message if ECH connectivity was not established.

Appendix L - DNS Records Management for ECH

The appendix is about DNS records which are required for ECH to be working, for the domain secret.ech-website.company. The www record will point to a non-ECH-enabled part of the website, allowing clients which do not support ECH to visit the website with the use of www subdomain. Firstly, a new subdomain is to be created, by the name of secret. This subdomain points to the web server previously configured but with additional data. Another line is to be added with a HTTPS record for the website. The number one, proceeding HTTPS record type, specifies the priority of the record and alpn="h2" informs the support of HTTPS/2. Previously created ECH key is to be written out where the string between -----BEGIN ECHCONFIG----- and -----END ECHCONFIG----- is of interest. This string is to be copied and input after the ech= data field, within quotation marks. For more details, see the configuration below for the full DNS records configuration.

```
$TTL    604800
@       IN      SOA     ns1.ech-website.company.
admin.ech-website.company. (
                            20240503         ; Serial
                            604800         ; Refresh
                            86400         ; Retry
                            2419200         ; Expire
                            604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.ech-website.company.
@       IN      A       70.34.223.34
ns1     IN      A       70.34.221.99
www     IN      A       70.34.223.34
secret  IN      A       70.34.223.34
secret  IN      HTTPS   1 . alpn="h2"
ech="AE3+DQBJKwAgACA+0tcypDsECHuyDYDlAQbejPWpe9El7QZ5+hNHHP0TXAAE
AAEAAQAaaGlkZGVuLmVjaC13ZWJzaXRlLmNvbXBhbnkAAA=="
```

Appendix M - DNS Zone Records Management for Response Policy Zone

This appendix shows a zone configuration for setting up DNS RPZ.

```
zone "rpz.local" {
     type master;
     file "/etc/bind/db.rpz.local";
     allow-query { "none"; };
     allow-transfer { "none"; };
};
```

Appendix N - DNS Records Management RPZ

The appendix intends to clarify in technical terms what DNS records are needed as well as defining ECH to be blocked or redirected to another website. The configuration below shows db.rpz.local managing records for RPZ.

```
; BIND reverse data file for empty rfc1918 zone
;
; DO NOT EDIT THIS FILE - it is used for multiple zones.
; Instead, copy it, edit named.conf, and use that copy.
;
$TTL 86400
@     IN    SOA   localhost. root.localhost. (
                            20240508        ; Serial
                       604800          ; Refresh
                            86400      ; Retry
                       2419200         ; Expire
                            86400 )    ; Negative Cache TTL
;
@     IN    NS    localhost.
secret.ech-website.company      A 70.34.201.112
tls-ech.dev     CNAME .
defo.ie         CNAME .
crypto.cloudflare.com           CNAME .
research.cloudflare.com         CNAME .
```

To explain the usage context of RPZ, CNAME with the website afterwards was redirecting the webpage. The dot after CNAME is defined as NXDOMAIN (ISC, n.d.). If there is a wildcard with the dot after CNAME, that means that there is NODATA for the domain, meaning no content is shown for the web page. There is a possibility of redirecting the client if a matching response policy is found. This is done by adding the IP address of a web server or a correlating name which is later translated locally. Redirection was only replicated to a HTTP website which will display an error on the browser regarding insecure connection due to invalid certificate if the requested website is specified to support HTTPS.

To reload changes on BIND9, restart the application by reloading configuration, then restarting BIND9, then checking status for any errors by following the commands below.

```
rndc reload
systemctl restart bind9
systemctl status bind9
```

Appendix O - RPZ Logging

Appendix O intends to show how to enable and configure RPZ logging on DNS RPZ. To enable RPZ logging capabilities, type in the shell nano /etc/bind/named.conf and append configuration parameters according to configuration below:

```
logging {
     channel rpzlog {
     file "/var/lib/bind/rpz.log" versions 3 size 3m;
     print-time yes;
     print-category yes;
     print-severity yes;
     severity info;
     //severity notice;
     };
     category rpz { rpzlog; };
};
```

To enable RPZ logging, the following command should be typed for making symbolic links to /var/log/ is sudo ln -s /var/lib/bind/ /var/log/. To view the logs, type the following command: tail /var/lib/bind/rpz.log. By making sure that configuration files are correct for BIND9 type the following command: named-checkconf. If no prompt was shown, then configuration was correctly set up.

Appendix P - nginx.conf

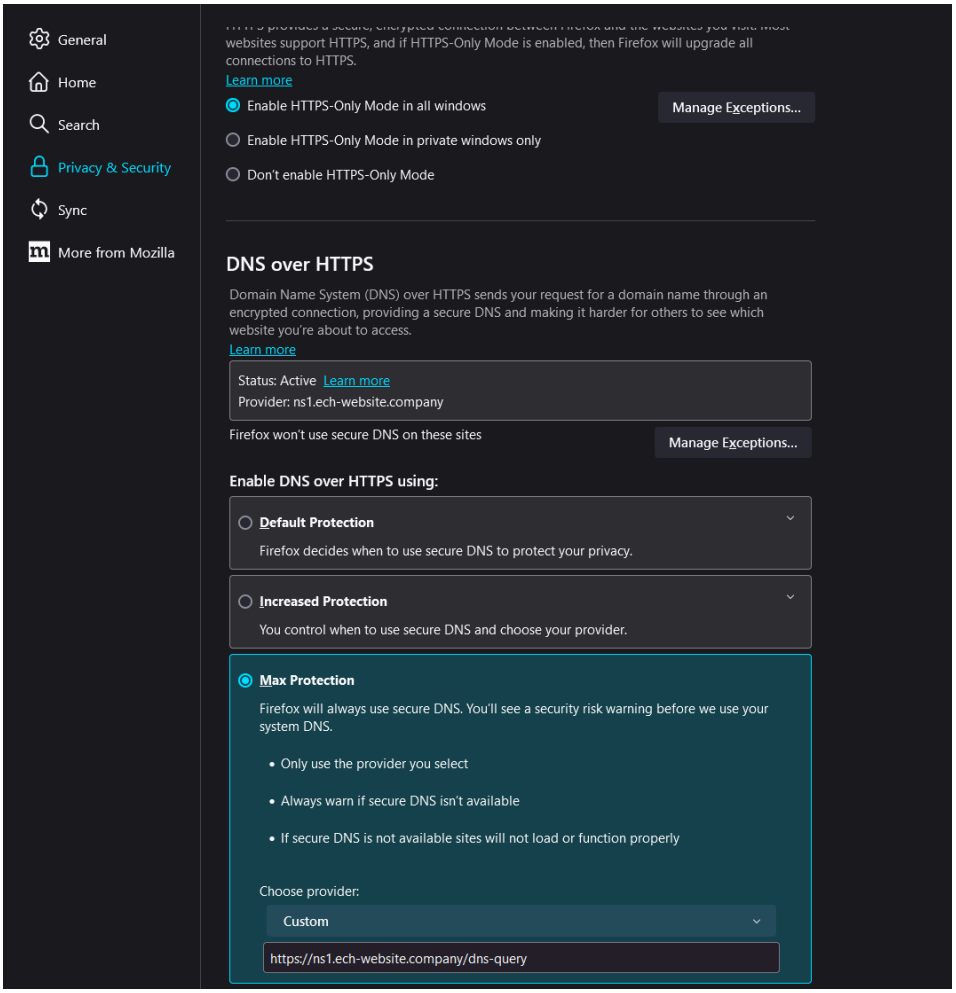
This configuration is the ECH web server configuration for the nginx. The configuration file is located at: /opt/ech/conf/nginx.conf.

```
worker_processes  1;
events {
     worker_connections  1024;
}
http {
# General configuration for Nginx
     include         mime.types;
     default_type    application/octet-stream;
     ssl_echkeydir   ech_keys;
     sendfile        on;
     keepalive_timeout  65;
     # HTTP
     server {
# Port specific configuration HTTP
     listen          80 default_server;
     server_name     localhost;
     location / {
          root       html;
          ssi        on;
          index      index.html index.htm;
          keepalive_timeout 0;
          # Force non-cache
          add_header Last-Modified $date_gmt;
          add_header Cache-Control 'no-store, no-cache';
          if_modified_since off;
          expires off;
          etag off;
     }
     error_page      500 502 503 504  /50x.html;
     location = /50x.html {
          root       html;
     }
     }
     # HTTPS
     server {
# Port specific configuration HTTPS
     listen               443 ssl;
     server_name          localhost;

     ssl_certificate
/etc/letsencrypt/live/ech-website.company/fullchain.pem;
     ssl_certificate_key
/etc/letsencrypt/live/ech-website.company/privkey.pem;
     ssl_session_cache    shared:SSL:1m;
     ssl_session_timeout  5m;
     ssl_ciphers          HIGH:!aNULL:!MD5;
```

```
      ssl_prefer_server_ciphers  on;
      location / {
            root   html;
                  ssi   on;
            index  index.html index.htm;
            keepalive_timeout 0;
            # Force non-cache
            add_header Last-Modified $date_gmt;
            add_header Cache-Control 'no-store, no-cache';
            if_modified_since off;
            expires off;
            etag off;
      }
      }
}
```

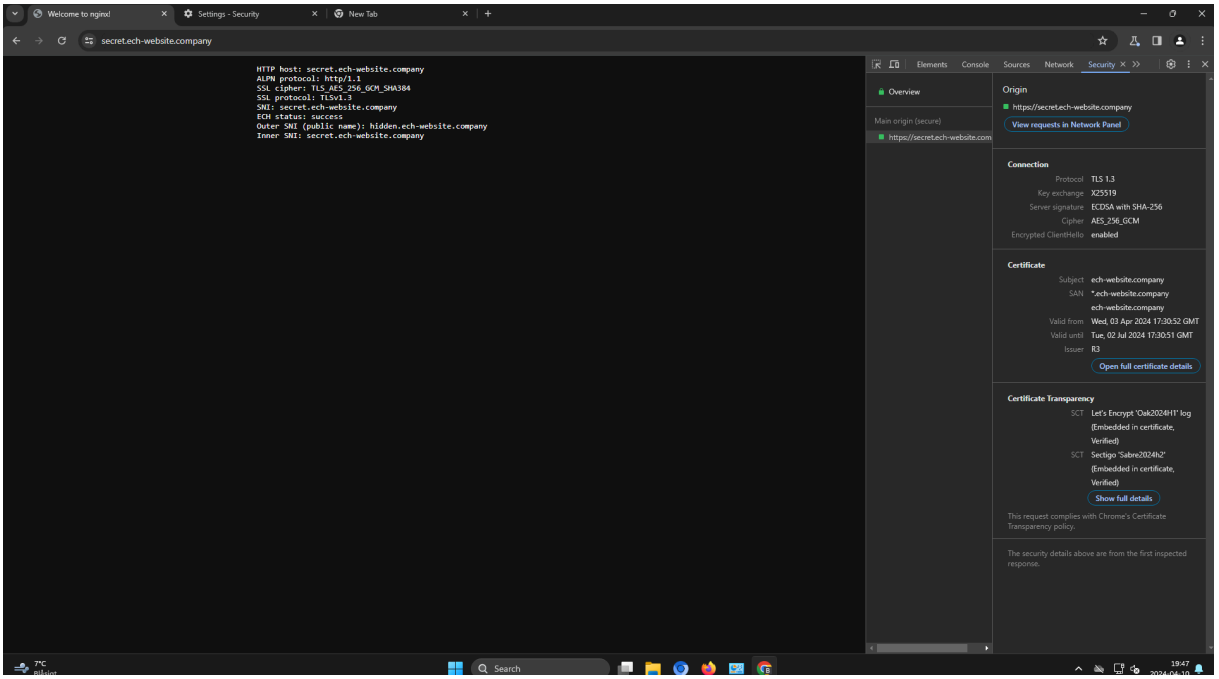
Appendix Q - DoH Client Configuration

DoH client configuration shows max protection with a custom DoH provider.



Appendix R - ECH enabled-Website Example

ECH enabled website with inspection proof with Google Chrome beta version with the own DoH provider configured in Firefox browser.



Appendix S - RPZ Configuration of named.conf.options

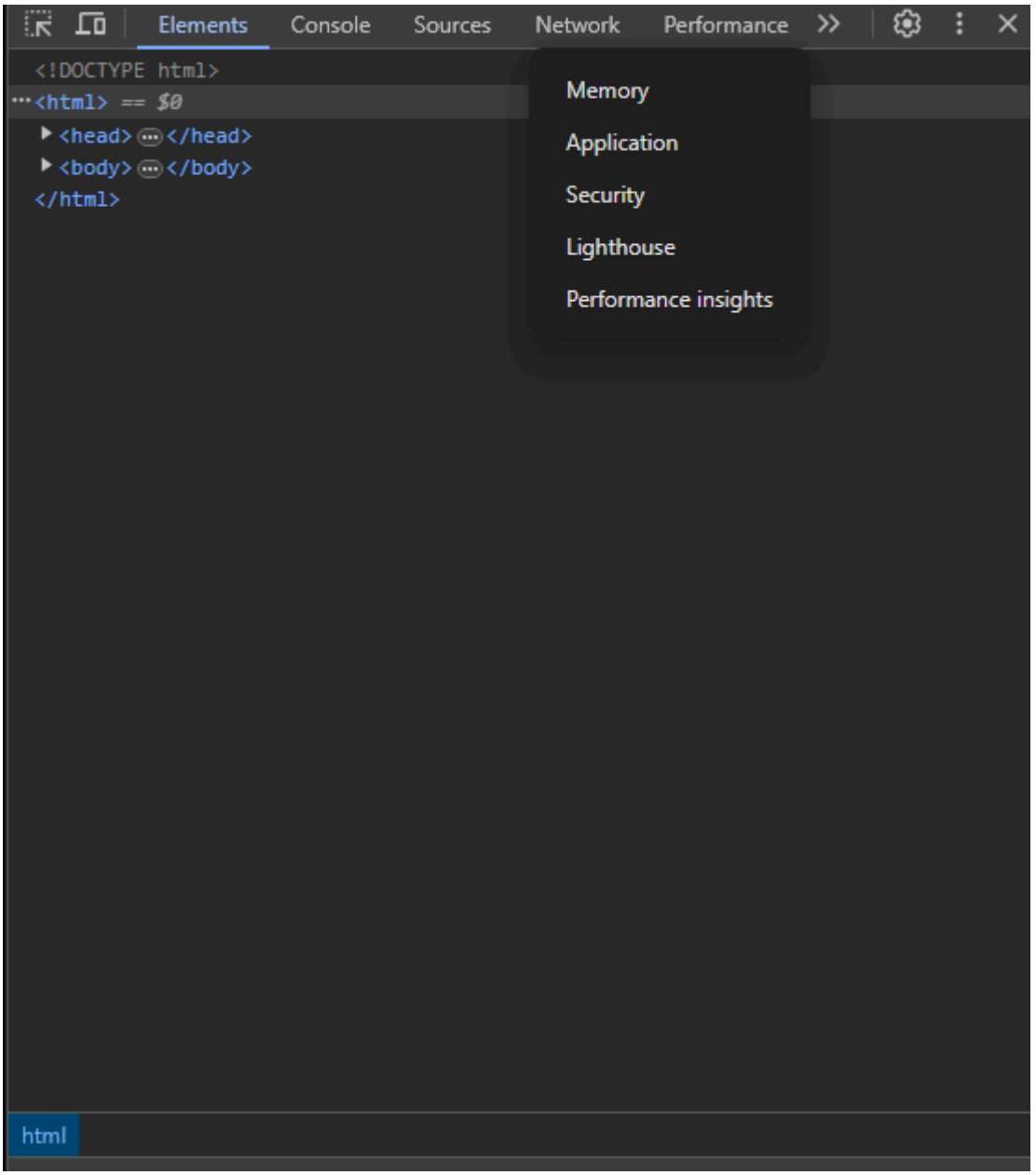
This configuration is a BIND9 configuration for RPZ.

```
tls local-tls {
     key-file
"/etc/letsencrypt/live/filter-website.company/privkey.pem";
     cert-file
"/etc/letsencrypt/live/filter-website.company/fullchain.pem";
};



http local-http-server {
#     multiple paths can be specified
     endpoints { "/dns-query";  };
};


options {
     response-policy {
          zone "rpz.local";
     };
     directory "/var/cache/bind";
     forwarders {
               70.34.221.99;
     };
     dnssec-validation auto;
     auth-nxdomain no; # conform to RFC1035
     listen-on-v6 { any; };
     allow-transfer { none; };
     allow-query { any; };
     allow-recursion { any; };
     #Doh
     listen-on port 53 { any; };
 #   http-port 80;
     https-port 443;
```

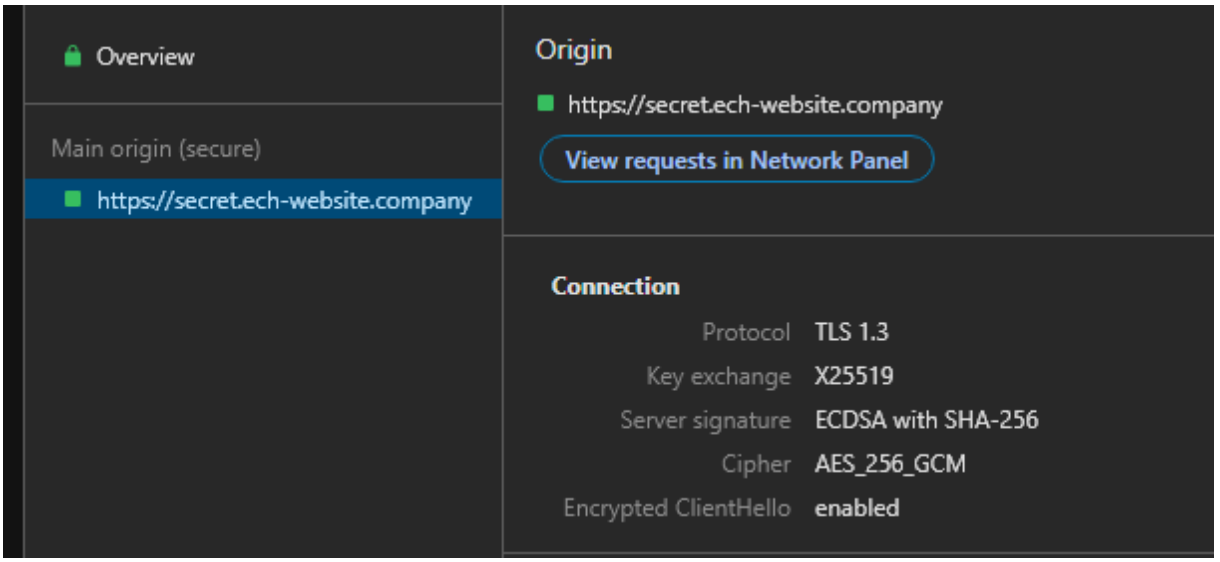
```
      #IPv4 sektion
      # example for encrypted and unencrypted DoH
      # listening on all IPv4 addresses.
      # port number can be omitted
      listen-on  port  443  tls  local-tls  http  local-http-server
{any;};
#     listen-on port 80 http local-http-server {any;};


};
```

Appendix T - Step-by-Step Guide on Obtaining "Encrypted ClientHello" Flag

- To obtain *Encrypted ClientHello* flag, head for the chosen Chromium browser, head to one of the ECH enabled websites → Right-click on the webpage screen and select *Inspect*
- In the tab section, click on two arrows to expand options to select *Security* option.



- Next, refresh the webpage.
- In the overview section, under the *Main origin (secure)* section, a visited website appears. Click on it to expand for more details.



- Under *Connection* on the right side, there is the flag *Encrypted ClientHello*, and it is *enabled*.
- This concludes the step-by-step guide on how to obtain *Encrypted ClientHello* on Chromium-based browsers.

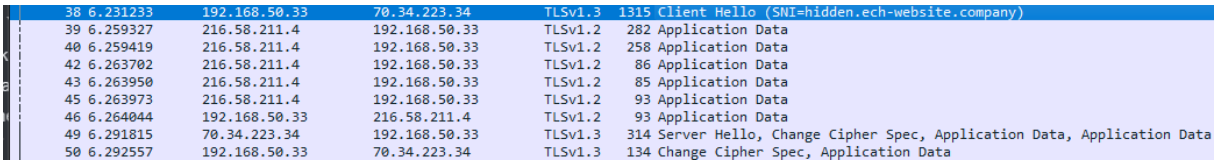
Appendix U - Division of Work

The table below represents the division of work of each segment. The numbers of each authors' contribution is measured in percentage (%). Each moment is conducted via in person, meetings, or online conference calls. Each author assigned to a task agreed on by both parties. Each author got involved in team working assignments of activities in the research project, as well as individual assignments when authors were working on individual assigned tasks.

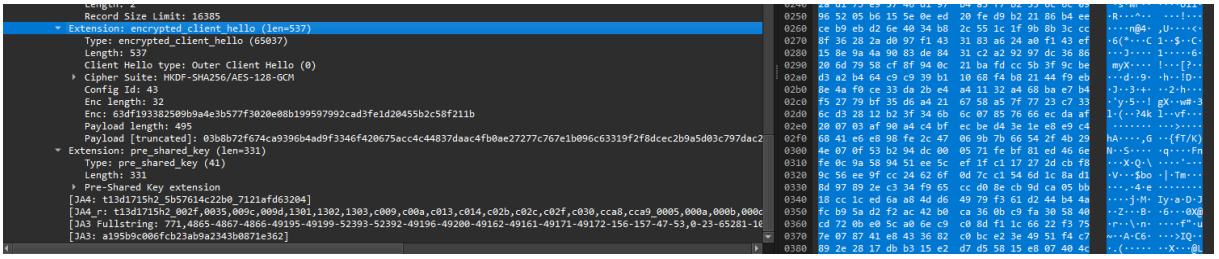
Activity	Paulius Contribution	Luka Contribution
Laboratory Implementation Research	50	50
Laboratory Configuration	50	50
Laboratory Verification	50	50
Report Writing	50	50
Report Academic Compliance Verification	50	50
Report Restructuring	50	50
Report Proofreading	50	50
Total Effort & Contribution	50	50

Appendix V - TLS ECH Handshakes in Wireshark

The first image shows the example of the TLS ECH handshake in Wireshark. While the second image is an example of the same TLS handshake in Wireshark. Handshake on the left column shows that ECH has been utilised and ECH values on the right side.



38	6.231233	192.168.50.33	70.34.223.34	TLSv1.3	1315 Client Hello (SNI=hidden.ech-website.company)
39	6.259327	216.58.211.4	192.168.50.33	TLSv1.2	282 Application Data
40	6.259419	216.58.211.4	192.168.50.33	TLSv1.2	258 Application Data
42	6.263702	216.58.211.4	192.168.50.33	TLSv1.2	86 Application Data
43	6.263950	216.58.211.4	192.168.50.33	TLSv1.2	85 Application Data
45	6.263973	216.58.211.4	192.168.50.33	TLSv1.2	93 Application Data
46	6.264044	192.168.50.33	216.58.211.4	TLSv1.2	93 Application Data
49	6.291815	70.34.223.34	192.168.50.33	TLSv1.3	314 Server Hello, Change Cipher Spec, Application Data, Application Data
50	6.292557	192.168.50.33	70.34.223.34	TLSv1.3	134 Change Cipher Spec, Application Data



```
    Record Size Limit: 16385
  Extension: encrypted_client_hello (len=537)
    Type: encrypted_client_hello (65037)
    Length: 537
    Client Hello type: Outer Client Hello (0)
  > Cipher Suite: HKDF-SHA256/AES-128-GCM
    Config Id: 43
    Enc length: 32
    Enc: 63df193382509b9a4e3b577f3020e08b199597992cad3fe1d20455b2c58f211b
    Payload length: 495
    Payload [truncated]: 03b8b72f674ca9396b4ad9f3346f420675acc4c44837daac4fb0ae27277c767e1b096c63319f2f8dcec2b9a5d03c797dac2
  Extension: pre_shared_key (len=331)
    Type: pre_shared_key (41)
    Length: 331
  > Pre-Shared Key extension
  [JA4: t13d1715h2_5b57614c22b0_7121afd63204]
  [JA4_r: t13d1715h2_002f,0035,009c,009d,1301,1302,1303,c009,c00a,c013,c014,c02b,c02c,c02f,c030,cca8,cca9_0005,000a,000b,000c
  [JA3 Fullstring: 771,4865-4867-4866-49195-49199-52393-52392-49196-49200-49162-49161-49171-49172-156-157-47-53,0-23-65281-16
  [JA3: a195b9c006fcb23ab9a2343b0871e362]
```

```
0250  96 52 05 b6 15 5e 0e ed  20 fe d9 b2 21 86 b4 ee   ·R···^·· ···!···
0260  ce b9 eb d2 6e 40 34 b8  2c 55 1c 1f 9b 8b 3c cc   ····n@4· ,U····<·
0270  8f 36 28 2a d0 97 f1 43  31 83 a6 24 a0 f1 43 ef   ·6(*···C 1··$··C·
0280  15 8e 9a 4a 90 83 de 84  31 c2 a2 92 97 dc 36 86   ···J···· 1·····6·
0290  20 6d 79 58 cf 8f 94 0c  21 ba fd cc 5b 3f 9c be    myX···· !···[?··
02a0  d3 a2 b4 64 c9 c9 39 b1  10 68 f4 b8 21 44 f9 eb   ···d··9· ·h··!D··
02b0  8e 4a f0 ce 33 da 2b e4  a4 11 32 a4 68 ba e7 b4   ·J··3·+· ··2·h···
02c0  f5 27 79 bf 35 d6 a4 21  67 58 a5 7f 77 23 c7 33   ·'y·5··! gX··w#·3
02d0  6c d3 28 12 b2 3f 34 6b  6c 07 85 76 66 ec da af   l·(··?4k l··vf···
02e0  20 07 03 af 90 a4 c4 bf  ec be d4 3e 1e e8 e9 c4    ······ ···>····
02f0  68 41 e6 e8 98 fe 2c 47  06 9b 7b 66 54 2f 4b 29   hA····,G ··{fT/K)
0300  4e 07 0f 53 b2 94 dc 00  05 71 fe bf 81 ed 46 6e   N··S···· ·q····Fn
0310  fe 0c 9a 58 94 51 ee 5c  ef 1f c1 17 27 2d cb f8   ···X·Q·\ ····'···
0320  9c 56 ee 9f cc 24 62 6f  0d 7c c1 54 6d 1c 8a d1   ·V···$bo ·|·Tm···
0330  8d 97 89 2e c3 34 f9 65  cc d0 8e cb 9d ca 05 bb   ·····4·e ········
0340  18 cc 1c ed 6a a8 4d d6  49 79 f3 61 d2 44 b4 4a   ····j·M· Iy·a·D·J
0350  fc b9 5a d2 f2 ac 42 b0  ca 36 0b c9 fa 30 58 40   ··Z···B· ·6···0X@
0360  cd 72 0b e0 5c a0 6e c9  c0 8d f1 1c 66 22 f3 75   ·r··\·n· ····f"·u
0370  7e 07 87 41 e8 43 36 82  c0 bc e2 3e 49 51 f4 c7   ~··A·C6· ···>IQ··
0380  89 2e 28 17 db b3 15 e2  d7 d5 58 15 e8 07 40 4c   ··(····· ··X···@L
```